

## UNIVERSAL NETWORK DRIVER INTERFACES

### E.1 Introduction

This appendix defines the 32/64-bit H/W and S/W Universal Network Driver Interfaces (UNDIs). These interfaces provide one method for writing a network driver; other implementations are possible.

#### E.1.1 Definitions

##### E-1 Definitions

Table E.1: Definitions

| Term | Definition                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| BC   | <p><b>BaseCode</b></p> <p>The PXE BaseCode, included as a core protocol in EFI, is comprised of a simple network stack (UDP/IP) and a few common network protocols (DHCP, Bootserver Discovery, TFTP) that are useful for remote booting machines.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| LOM  | <p><b>LAN On Motherboard</b></p> <p>This is a network device that is built onto the motherboard (or baseboard) of the machine.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| NBP  | <p><b>Network Bootstrap Program</b></p> <p>This is the first program that is downloaded into a machine that has selected a PXE capable device for remote boot services.</p> <p>A typical NBP examines the machine it is running on to try to determine if the machine is capable of running the next layer (OS or application).</p> <p>If the machine is not capable of running the next layer, control is returned to the EFI boot manager and the next boot device is selected.</p> <p>If the machine is capable, the next layer is downloaded and control can then be passed to the downloaded program.</p> <p>Though most NBPs are OS loaders, NBPs can be written to be standalone applications such as diagnostics, backup/restore, remote management agents, browsers, etc.</p> |

continues on next page

Table E.1 – continued from previous page

|      |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| NIC  | <p><b>Network Interface Card</b></p> <p>Technically, this is a network device that is inserted into a bus on the motherboard or in an expansion board. For the purposes of this document, the term NIC will be used in a generic sense, meaning any device that enables a network connection (including LOMs and network devices on external busses (USB, 1394, etc.)).</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| ROM  | <p><b>Read-Only Memory</b></p> <p>When used in this specification, ROM refers to a nonvolatile memory storage device on a NIC.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| PXE  | <p><b>Preboot Execution Environment</b></p> <p>The complete PXE specification covers three areas; the client, the network and the server.</p> <p><b>Client</b></p> <ul style="list-style-type: none"> <li>• Makes network devices into bootable devices.</li> <li>• Provides APIs for PXE protocol modules in EFI and for universal drivers in the OS.</li> </ul> <p><b>Network</b></p> <ul style="list-style-type: none"> <li>• Uses existing technology: DHCP, TFTP, etc.</li> <li>• Adds “vendor specific” tags to DHCP to define PXE specific operation within DHCP.</li> <li>• Adds multicast TFTP for high bandwidth remote boot applications.</li> <li>• Defines Bootserver discovery based on DHCP packet format.</li> </ul> <p><b>Server</b></p> <ul style="list-style-type: none"> <li>• <b>Bootserver:</b> Responds to Bootserver discovery requests and serves up remote boot images.</li> <li>• <b>proxyDHCP:</b> Used to ease the transition of PXE clients and servers into existing network infrastructure. proxyDHCP provides the additional DHCP information that is needed by PXE clients and Bootservers without making changes to existing DHCP servers.</li> <li>• <b>MTFTP:</b> Adds multicast support to a TFTP server.</li> <li>• <b>Plug-In Modules:</b> Example proxyDHCP and Bootservers provided in the PXE SDK (software development kit) have the ability to take plug-in modules (PIMs). These PIMs are used to change/enhance the capabilities of the proxyDHCP and Bootservers.</li> </ul> |
| UNDI | <p><b>Universal Network Device Interface</b></p> <p>UNDI is an architectural interface to NICs. Traditionally NICs have had custom interfaces and custom drivers (each NIC had a driver for each OS on each platform architecture).</p> <p>Two variations of UNDI are defined in this specification: H/W UNDI and S/W UNDI. H/W UNDI is an architectural hardware interface to a NIC. S/W UNDI is a software implementation of the H/W UNDI.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |

## E.1.2 Referenced Specifications

When implementing PXE services, protocols, ROMs or drivers, it is a good idea to understand the related network protocols and BIOS specifications. The Table, below includes all of the specifications referenced in this document.

### Referenced Specifications

Table E.2: Referenced Specifications

| Acronym                 | Protocol/Specification                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|-------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>ARP</b>              | <p>Address Resolution Protocol</p> <ul style="list-style-type: none"> <li>• Required reading for those implementing the PXE Base Code Protocol.</li> <li>• See “Links to UEFI-Related Documents” (<a href="http://uefi.org/uefi">http://uefi.org/uefi</a>) under the heading “Address Resolution Protocol”.</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| <b>Assigned Numbers</b> | <p>Lists the reserved numbers used in the RFCs and in this specification.</p> <ul style="list-style-type: none"> <li>• See “Links to UEFI-Related Documents” (<a href="http://uefi.org/uefi">http://uefi.org/uefi</a>) under the heading “Assigned Numbers”.</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| <b>BIOS</b>             | <p>Basic Input/Output System</p> <ul style="list-style-type: none"> <li>• Contact your BIOS manufacturer for reference and programming manuals.</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| <b>BOOTP</b>            | <p><b>Bootstrap Protocol -</b></p> <ul style="list-style-type: none"> <li>• See “Links to UEFI-Related Documents” (<a href="http://uefi.org/uefi">http://uefi.org/uefi</a>) under the heading “Bootstrap Protocol (BOOTP)”.</li> </ul> <p>These references are included for backward compatibility. BC protocol supports DHCP and BOOTP:</p> <ul style="list-style-type: none"> <li>• See “Links to UEFI-Related Documents” (<a href="http://uefi.org/uefi">http://uefi.org/uefi</a>) under the heading “BOOTP Clarifications and Extensions”.</li> <li>• See “Links to UEFI-Related Documents” (<a href="http://uefi.org/uefi">http://uefi.org/uefi</a>) under the heading “Bootstrap Protocol (BOOTP) Interoperation Between DHCP and BOOTP”.</li> </ul> <p>Required reading for those implementing the PXE Base Code Protocol BC protocol or PXE Bootservers.</p> |

continues on next page

Table E.2 – continued from previous page

## DHCP

### Dynamic Host Configuration Protocol

- See “Links to UEFI-Related Documents” (<http://uefi.org/uefi>) under the heading “DHCP”.
- See “Links to UEFI-Related Documents” (<http://uefi.org/uefi>) under the heading “Index of RFC (IETF)”.
- See “Links to UEFI-Related Documents” (<http://uefi.org/uefi>) under the heading “DHCP Reconfigure Extension”.
- See “Links to UEFI-Related Documents” (<http://uefi.org/uefi>) under the heading “DHCP for Ipv4”.
- See “Links to UEFI-Related Documents” (<http://uefi.org/uefi>) under the heading “Interoperations between DHCP and BOOTP”.

Required reading for those implementing the PXE Base Code Protocol or PXE Bootservers.

## EFI

### Extensible Firmware Interface

- See “Links to UEFI-Related Documents” (<http://uefi.org/uefi>) under the heading “Intel Developer Centers”.

Required reading for those implementing NBPs, OS loaders and preboot applications for machines with the EFI preboot environment.

## ICMP

### Internet Control Message Protocol

- See “Links to UEFI-Related Documents” (<http://uefi.org/uefi>) under the heading “ICMP for Ipv4”.
- See “Links to UEFI-Related Documents” (<http://uefi.org/uefi>) under the heading “ICMP for Ipv6”. Required reading for those implementing the BC protocol.

## IETF

### Internet Engineering Task Force

See “Links to UEFI-Related Documents” (<http://uefi.org/uefi>) under the heading “Internet Engineering Task Force (IETF)”.

This is a good starting point for obtaining electronic copies of Internet standards, drafts, and RFCs.

## IGMP

### Internet Group Management Protocol

- See “Links to UEFI-Related Documents” (<http://uefi.org/uefi>) under the heading “Internet Group Management Protocol”.

Required reading for those implementing the PXE Base Code Protocol.

continues on next page

Table E.2 – continued from previous page

|              |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|--------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>IP</b>    | <p><b>Internet Protocol</b></p> <p>Ipv4: <a href="http://www.ietf.org/rfc/rfc0791.txt">http://www.ietf.org/rfc/rfc0791.txt</a></p> <ul style="list-style-type: none"> <li>• “Links to UEFI-Related Documents” (<a href="http://uefi.org/uefi">http://uefi.org/uefi</a>) under the heading “Ipv4”.</li> <li>• See “Links to UEFI-Related Documents” (<a href="http://uefi.org/uefi">http://uefi.org/uefi</a>) under the heading “Ipv6”.</li> </ul> <p>Required reading for those implementing the BC protocol.</p> |
| <b>MTFTP</b> | <p><b>Multicast TFTP</b> - Defined in the 16-bit PXE specification.</p> <p>Required reading for those implementing the PXE Base Code Protocol.</p>                                                                                                                                                                                                                                                                                                                                                                |
| <b>PCI</b>   | <p><b>Peripheral Component Interface</b></p> <ul style="list-style-type: none"> <li>• See “Links to UEFI-Related Documents” (<a href="http://uefi.org/uefi">http://uefi.org/uefi</a>) under the heading “Peripheral Component Interface (PCI)”.</li> </ul> <p>Source for PCI specifications. Required reading for those implementing S/W or H/W UNDI on a PCI NIC or LOM.</p>                                                                                                                                     |
| <b>PnP</b>   | <p><b>Plug-and-Play</b></p> <ul style="list-style-type: none"> <li>• <a href="http://www.phoenix.com/en/support/white+papers-specs/">http://www.phoenix.com/en/support/white+papers-specs/</a></li> <li>• See “Links to UEFI-Related Documents” (<a href="http://uefi.org/uefi">http://uefi.org/uefi</a>) under the heading “Plug and Play”.</li> </ul> <p>Source for PnP specifications.</p>                                                                                                                     |
| <b>PXE</b>   | <p><b>Preboot eXecution Environment</b></p> <p>16-bit PXE v2.1:</p> <ul style="list-style-type: none"> <li>• See “Links to UEFI-Related Documents” (<a href="http://uefi.org/uefi">http://uefi.org/uefi</a>) under the heading “Preboot eXecution Environment (PXE)”.</li> </ul> <p>Required reading.</p>                                                                                                                                                                                                         |
| <b>RFC</b>   | <p><b>Request For Comments</b></p> <ul style="list-style-type: none"> <li>• See “Links to UEFI-Related Documents” (<a href="http://uefi.org/uefi">http://uefi.org/uefi</a>) under the heading “Request for Comments”.</li> </ul>                                                                                                                                                                                                                                                                                  |
| <b>TCP</b>   | <p><b>Transmission Control Protocol</b></p> <ul style="list-style-type: none"> <li>• See “Links to UEFI-Related Documents” (<a href="http://uefi.org/uefi">http://uefi.org/uefi</a>) under the heading “TCPv4”.</li> <li>• See “Links to UEFI-Related Documents” (<a href="http://uefi.org/uefi">http://uefi.org/uefi</a>) under the heading “TCPv6”.</li> </ul> <p>Required reading for those implementing the PXE Base Code Protocol .</p>                                                                      |

continues on next page

Table E.2 – continued from previous page

|      |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| TFTP | <p><b>Trivial File Transfer Protocol</b></p> <p>TFTP</p> <ul style="list-style-type: none"> <li>• See “Links to UEFI-Related Documents” (<a href="http://uefi.org/uefi">http://uefi.org/uefi</a>) under the heading “TFTP Protocol”.</li> <li>• See “Links to UEFI-Related Documents” (<a href="http://uefi.org/uefi">http://uefi.org/uefi</a>) under the heading “TFTP Option Extension”.</li> <li>• See “Links to UEFI-Related Documents” (<a href="http://uefi.org/uefi">http://uefi.org/uefi</a>) under the heading “TFTP Blocksize Option”.</li> <li>• See “Links to UEFI-Related Documents” (<a href="http://uefi.org/uefi">http://uefi.org/uefi</a>) under the heading “TFTP Timeout Interval and Transfer Size Options”.</li> </ul> <p>Required reading for those implementing the PXE Base Code Protocol.</p> |
| UDP  | <p><b>User Datagram Protocol</b></p> <ul style="list-style-type: none"> <li>• See “Links to UEFI-Related Documents” (<a href="http://uefi.org/uefi">http://uefi.org/uefi</a>) under the heading “UDP over IPv4”.</li> <li>• See “Links to UEFI-Related Documents” (<a href="http://uefi.org/uefi">http://uefi.org/uefi</a>) under the heading “UDP over IPv6”.</li> </ul> <p>Required reading for those implementing the PXE Base Code Protocol.</p>                                                                                                                                                                                                                                                                                                                                                                   |

### E.1.3 OS Network Stacks

This is a simplified overview of three OS network stacks that contain three types of network drivers: Custom, S/W UNDI and H/W UNDI. The Figure, below, depicts an application bound to an OS protocol stack, which is in turn bound to a protocol driver that is bound to three NICs. The Table, below, *Driver Types: Pros and Cons* gives a brief list of pros and cons about each type of driver implementation.

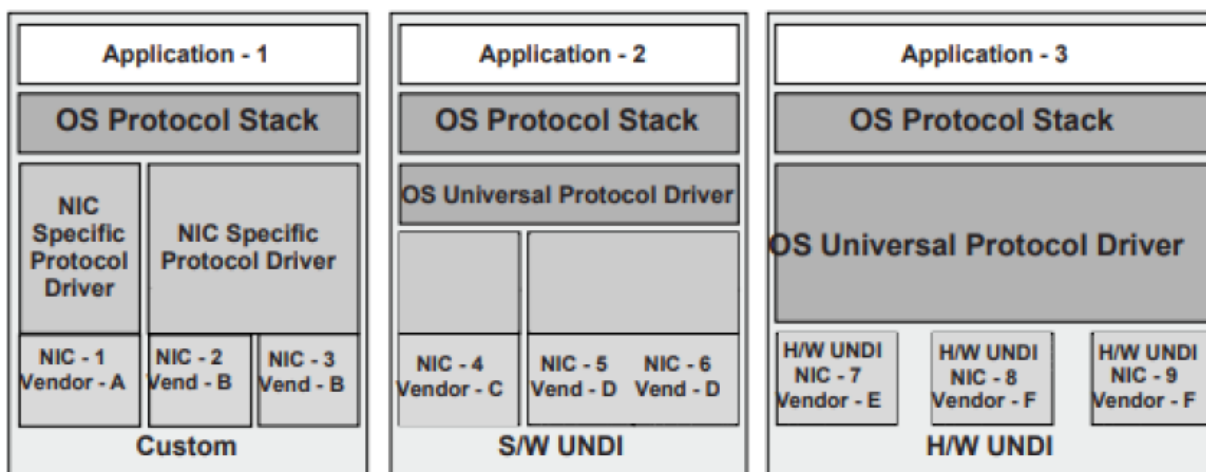


Fig. E.1: Network Stacks with Three Classes of Drivers

#### Driver Types: Pros and Cons

Table E.3: Driver Types: Pros and Cons

| Driver          | Pro                                                                                                                                                                                                                                                                                                                                                             | Con                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Custom</b>   | <ul style="list-style-type: none"> <li>• Can be very fast and efficient. NIC vendor tunes driver to OS &amp; device.</li> <li>• OS vendor does not have to write NIC driver.</li> </ul>                                                                                                                                                                         | <ul style="list-style-type: none"> <li>• New driver for each OS/architecture must be maintained by NIC vendor.</li> <li>• OS vendor must trust code supplied by third-party.</li> <li>• OS vendor cannot test all possible driver/NIC versions.</li> <li>• Driver must be installed before NIC can be used.</li> <li>• Possible performance sink if driver is poorly written.</li> <li>• Possible security risk if driver has back door.</li> </ul> |
| <b>S/W UNDI</b> | <ul style="list-style-type: none"> <li>• S/W UNDI driver is simpler than a Custom driver. Easier to test outside of the OS environment.</li> <li>• OS vendor can tune the universal protocol driver for best OS performance.</li> <li>• NIC vendor only has to write one driver per processor architecture.</li> </ul>                                          | <ul style="list-style-type: none"> <li>• Slightly slower than Custom or H/W UNDI because of extra call layer between protocol stack and NIC.</li> <li>• S/W UNDI driver must be loaded before NIC can be used.</li> <li>• OS vendor has to write the universal driver.</li> <li>• Possible performance sink if driver is poorly written.</li> <li>• Possible security risk if driver has back door.</li> </ul>                                      |
| <b>H/W UNDI</b> | <ul style="list-style-type: none"> <li>• H/W UNDI provides a common architectural interface to all network devices.</li> <li>• OS vendor controls all security and performance issues in network stack.</li> <li>• NIC vendor does not have to write any drivers.</li> <li>• NIC can be used without an OS or driver installed (preboot management).</li> </ul> | <ul style="list-style-type: none"> <li>• OS vendor has to write the universal driver (this might also be a Pro, depending on your point of view).</li> </ul>                                                                                                                                                                                                                                                                                        |

## E.2 Overview

There are three major design changes between this specification and the 16-bit UNDI in version 2.1 of the PXE Specification:

- A new architectural hardware interface has been added.
- All UNDI commands use the same command format.
- BC is no longer part of the UNDI ROM.

## E.2.1 32/64-bit UNDI Interface

The !PXE structures are used to locate and identify the type of 32/64-bit UNDI interface (H/W or S/W), as shown in the Figure, below. These structures are normally only used by the system BIOS and universal network drivers.

**Major Minor reserved**

| !PXE<br>H/W UNDI |                |       |          |          | !PXE<br>S/W UNDI |                  |       |          |          |
|------------------|----------------|-------|----------|----------|------------------|------------------|-------|----------|----------|
| Offset           | 0x00           | 0x01  | 0x02     | 0x03     | Offset           | 0x00             | 0x01  | 0x02     | 0x03     |
| 0x00             | Signature      |       |          |          | 0x00             | Signature        |       |          |          |
| 0x04             | Len            | Fudge | Rev      | IFcnt    | 0x04             | Len              | Fudge | Rev      | IFcnt    |
| 0x08             | Major          | Minor | IFcntExt | Reserved | 0x08             | Major            | Minor | IFcntExt | Reserved |
| 0x0C             | Implementation |       |          |          | 0x0C             | Implementation   |       |          |          |
| 0x10             | reserved       |       |          |          | 0x10             | Entry Point      |       |          |          |
| Len              | Status         |       |          |          | 0x14             |                  |       |          |          |
| Len + 0x04       | Command        |       |          |          | 0x18             | reserved         |       |          | #bus     |
| Len + 0x08       | CDBaddr        |       |          |          | 0x1C             | BusTypes(s)      |       |          |          |
| Len + 0x0C       |                |       |          |          | 0x20             | More BusTypes(s) |       |          |          |

Fig. E.2: !PXE Structures for H/W and S/W UNDI

The !PXE structures used for H/W and S/W UNDI are similar but not identical. The difference in the format is tied directly to the differences required by the implementation. The !PXE structures for 32/64-bit UNDI are not compatible with the !PXE structure for 16-bit UNDI.

The !PXE structure for H/W UNDI is built into the NIC hardware. The first nine fields (from offsets 0x00 to 0x0F) are implemented as read-only memory (or ports). The last three fields (from Len to Len + 0x0F) are implemented as read/write memory (or ports). The optional reserved field at 0x10 is not defined in this specification and may be used for vendor data.

The !PXE structure for S/W UNDI can be loaded into system memory from one of three places; ROM on a NIC, system nonvolatile storage, or external storage. Since there are no direct memory or I/O ports available in the S/W UNDI !PXE structure, an indirect callable entry point is provided. S/W UNDI developers are free to make their internal designs as simple or complex as they desire, as long as all of the UNDI commands in this specification are implemented. Descriptions of the fields in the Table below.

Table E.4: !PXE Structure Field Definitions

| Identifier | Value  | Description                                                                                                                                                                                                                                                                                                                       |
|------------|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Signature  | “!PXE” | !PXE structure signature. This field is used to locate an UNDI hardware or software interface in system memory (or I/O) space. ‘!’ is in the first (lowest address) byte, ‘P’ is in the second byte, ‘X’ in the third and ‘E’ in the last. This field must be aligned on a 16-byte boundary (the last address byte must be zero). |

continues on next page



Table E.4 – continued from previous page

|                 |               |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|-----------------|---------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Len</b>      | <b>Varies</b> | <p>Number of !PXE structure bytes to checksum.</p> <p>When computing the checksum of this structure the Len field MUST be used as the number of bytes to checksum.</p> <p>The !PXE structure checksum is computed by adding all of the bytes in the structure, starting with the first byte of the structure Signature: '!'.<br/>If the 8-bit sum of all of the unsigned bytes in this structure is not zero, this is not a valid !PXE structure.</p>                                                                                                                    |
| <b>Fudge</b>    | <b>Varies</b> | This field is used to make the 8-bit checksum of this structure equal zero.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| <b>Rev</b>      | <b>0x03</b>   | Revision of this structure.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| <b>IFcnt</b>    | <b>Varies</b> | <p>This field reports the number (minus one) of physical external network connections that are controlled by this !PXE interface. (If there is one network connector, this field is zero.</p> <p>If there are two network connectors, this field is one.)</p> <p>For !PXE structure revision 0x03 or higher, in addition to this field, the value in IFcntExt field must be left-shifted by 8-bits and ORed with IFcnt to get the 16-bit value for the total number (minus one) of physical external network connections that are controlled by this !PXE interface.</p> |
| <b>Major</b>    | <b>Varies</b> | <p>UNDI command interface. Minor revision number.</p> <p><b>0x00 (Alpha):</b> This version of UNDI does not operate as a runtime driver. The callback interface defined in the UNDI Start command is required.</p> <p><b>0x10 (Beta):</b> This version of UNDI can operate as an OS runtime driver. The callback interface defined in the UNDI Start command is required</p>                                                                                                                                                                                             |
| <b>Minor</b>    | <b>Varies</b> | <p>UNDI command interface. Minor revision number.</p> <p><b>0x00 (Alpha):</b> This version of UNDI does not operate as a runtime driver. The callback interface defined in the UNDI Start command is required.</p> <p><b>0x10 (Beta):</b> This version of UNDI can operate as an OS runtime driver. The callback interface defined in the UNDI Start command is required.</p>                                                                                                                                                                                            |
| <b>IFcntExt</b> | <b>Varies</b> | <p>If the !PXE structure revision 0x02 or earlier, this field is reserved and must be set to zero.</p> <p>If the !PXE structure revision 0x03 or higher, this field reports the upper 8-bits of the number of physical external network connections that is controlled by this !PXE interface.</p>                                                                                                                                                                                                                                                                       |
| <b>Reserved</b> | <b>0x00</b>   | This field is reserved and must be set to zero.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |

continues on next page

Table E.4 – continued from previous page

| Implementation  | Varies   | <p>Identifies type of UNDI (S/W or H/W, 32 bit or 64 bit) and what features have been implemented. The implementation bits are defined below. Undefined bits must be set to zero by UNDI implementers. Applications/drivers must not rely on the contents of undefined bits (they may change later revisions).</p> <p>Bit 0x00: Command completion interrupts supported (1) or not supported (0)</p> <p>Bit 0x01: Packet received interrupts supported (1) or not supported (0)</p> <p>Bit 0x02: Transmit complete interrupts supported (1) or not supported (0)</p> <p>Bit 0x03: Software interrupt supported (1) or not supported (0)</p> <p>Bit 0x04: Filtered multicast receives supported (1) or not supported (0)</p> <p>Bit 0x05: Broadcast receives supported (1) or not supported (0)</p> <p>Bit 0x06: Promiscuous receives supported (1) or not supported (0)</p> <p>Bit 0x07: Promiscuous multicast receives supported (1) or not supported (0)</p> <p>Bit 0x08: Station MAC address settable (1) or not settable (0)</p> <p>Bit 0x09: Statistics supported (1) or not supported (0)</p> <p>Bit 0x0A,0x0B: NvData not available (0), read only (1), sparse write supported (2), bulk write supported (3)</p> <p>Bit 0x0C: Multiple frames per command supported (1) or not supported (0)</p> <p>Bit 0x0D: Command queuing supported (1) or not supported (0) Bit 0x0E: Command linking supported (1) or not supported (0)</p> <p>Bit 0x0F: Packet fragmenting supported (1) or not supported (0)</p> <p>Bit 0x10: Device can address 64 bits (1) or only 32 bits (0)</p> <p>Bit 0x1E: S/W UNDI: Entry point is virtual address (1) or unsigned offset from start of !PXE structure (0).</p> <p>Bit 0x1F: Interface type: H/W UNDI (1) or S/W UNDI (0)</p> |
|-----------------|----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| H/W UNDI Fields | Reserved | <p>This field is optional and may be used for OEM &amp; vendor unique data. If this field is present its length must be a multiple of 16 bytes and must be included in the !PXE structure checksum.</p> <p>This field, if present, will always start on a 16-byte boundary. Note: The size/contents of the !PXE structure may change in future revisions of this specification.</p> <p>Do not rely on OEM &amp; vendor data starting at the same offset from the beginning of the !PXE structure.</p> <p>It is recommended that the OEM &amp; vendor data include a signature that drivers/applications can search for.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |

continues on next page

Table E.4 – continued from previous page

| Status  | Varies                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|---------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|         | <p>UNDI operation, command and interrupt status flags.</p> <p>This is a read-only port. Undefined status bits must be set to zero. Reading this port does NOT clear the status.</p> <p>Bit 0x00: Command completion interrupt pending (1) or not pending (0)</p> <p>Bit 0x01: Packet received interrupt pending (1) or not pending (0)</p> <p>Bit 0x02: Transmit complete interrupt pending (1) or not pending (0)</p> <p>Bit 0x03: Software interrupt pending (1) or not pending (0)</p> <p>Bit 0x04: Command completion interrupts enabled (1) or disabled (0)</p> <p>Bit 0x05: Packet receive interrupts enabled (1) or disabled (0)</p> <p>Bit 0x06: Transmit complete interrupts enabled (1) or disabled (0)</p> <p>Bit 0x07: Software interrupts enabled (1) or disabled (0)</p> <p>Bit 0x08: Unicast receive enabled (1) or disabled (0)</p> <p>Bit 0x09: Filtered multicast receive enabled (1) or disabled (0)</p> <p>Bit 0x0A: Broadcast receive enabled (1) or disabled (0)</p> <p>Bit 0x0B: Promiscuous receive enabled (1) or disabled (0)</p> <p>Bit 0x0C: Promiscuous multicast receive enabled (1) or disabled (0)</p> <p>Bit 0x1D: Command failed (1) or command succeeded (0)</p> <p>Bits 0x1F:0x1E: UNDI state: Stopped (0), Started (1), Initialized (2), Busy (3)</p> |
| Command | Varies                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|         | <p>Use to execute commands, clear interrupt status and enable/disable receive levels.</p> <p>This is a read/write port. Read reflects the last write.</p> <p>Bit 0x00: Clear command completion interrupt (1) or NOP (0)</p> <p>Bit 0x01: Clear packet received interrupt (1) or NOP (0)</p> <p>Bit 0x02: Clear transmit complete interrupt (1) or NOP (0)</p> <p>Bit 0x03: Clear software interrupt (1) or NOP (0)</p> <p>Bit 0x04: Command completion interrupt enable (1) or disable (0)</p> <p>Bit 0x05: Packet receive interrupt enable (1) or disable (0)</p> <p>Bit 0x06: Transmit complete interrupt enable (1) or disable (0)</p> <p>Bit 0x07: Software interrupt enable (1) or disable (0). Setting this bit to (1) also generates a software interrupt.</p> <p>Bit 0x08: Unicast receive enable (1) or disable (0)</p> <p>Bit 0x09: Filtered multicast receive enable (1) or disable (0)</p> <p>Bit 0x0A: Broadcast receive enable (1) or disable (0)</p> <p>Bit 0x0B: Promiscuous receive enable (1) or disable (0)</p> <p>Bit 0x0C: Promiscuous multicast receive enable (1) or disable (0)</p> <p>Bit 0x1F: Operation type: Clear interrupt and/or filter (0), Issue command (1)</p>                                                                                         |

continues on next page

Table E.4 – continued from previous page

|                        |               |                                                                                                                                                                                                                                                                                                                                                                                        |
|------------------------|---------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>CDBAddr</b>         | <b>Varies</b> | <p>Write the physical address of a CDB to this port. (Done with one 64-bit or two 32-bit writes, depending on processor architecture.)</p> <p>When done, use one 32-bit write to the command port to send this address into the command queue.</p> <p>Unused upper address bits must be set to zero.</p>                                                                               |
| <b>S/W UNDI Fields</b> |               |                                                                                                                                                                                                                                                                                                                                                                                        |
| <b>EntryPoint</b>      | <b>Varies</b> | <p>S/W UNDI API entry point address.</p> <p>This is either a virtual address or an offset from the start of the !PXE structure. Protocol drivers will push the 64-bit virtual address of a CDB on the stack and then call the UNDI API entry point.</p> <p>When control is returned to the protocol driver, the protocol driver must remove the address of the CDB from the stack.</p> |
| <b>Reserved</b>        | <b>Zero</b>   | Reserved for future use.                                                                                                                                                                                                                                                                                                                                                               |
| <b>BusTypeCnt</b>      | <b>Varies</b> | This field is the count of 4-byte BusType entries in the next field.                                                                                                                                                                                                                                                                                                                   |
| <b>BusType</b>         | <b>Varies</b> | <p>This field defines the type of bus S/W UNDI is written to support: “PCIR,” “PCCR,” “USB” or “1394.” This field is formatted like the Signature field.</p> <p>If the S/W UNDI supports more than one BusType there will be more than one BusType identifier in this field.</p>                                                                                                       |

### E.2.1.1 Issuing UNDI Commands

How commands are written and status is checked varies a little depending on the type of UNDI (H/W or S/W) implementation being used. The command flowchart shown in the Figure, below, is a high-level diagram on how commands are written to both H/W and S/W UNDI.

### E.2.2 UNDI Command Format

The format of the CDB is the same for all UNDI commands. The Figure, below, shows the structure of the CDB. Some of the commands do not use or always require the use of all of the fields in the CDB. When fields are not used they must be initialized to zero or the UNDI will return an error. The StatCode and StatFlags fields must always be initialized to zero or the UNDI will return an error. All reserved fields (and bit fields) must be initialized to zero or the UNDI will return an error. Basically, the rule is: Do it right, or don’t do it at all.

Descriptions of the CDB fields are shown in the table below.

Table E.5: UNDI CDB Field Definitions

| Identifier | Description |
|------------|-------------|
|------------|-------------|

continues on next page

Table E.5 – continued from previous page

|         |                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|---------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| OpCode  | <p><b>Operation Code</b> (Function Number, Command Code, etc.)</p> <p>This field is used to identify the command being sent to the UNDI. The meanings of some of the bits in the OpFlags and StatFlags fields, and the format of the CPB and DB structures depends on the value in the OpCode field. Commands sent with an OpCode value that is not defined in this specification will not be executed and will return a StatCode of PXE_STATCODE_INVALID_CDB.</p> |
| OpFlags | <p><b>Operation Flags</b></p> <p>This bit field is used to enable/disable different features in a specific command operation. It is also used to change the format/contents of the CPB and DB structures. Commands sent with reserved bits set in the OpFlags field will not be executed and will return a StatCode of PXE_STATCODE_INVALID_CDB.</p>                                                                                                               |
| CPBsize | <p><b>Command Parameter Block Size</b></p> <p>This field should be set to a number that is equal to the number of bytes that will be read from CPB structure during command execution. Setting this field to a number that is too small will cause the command to not be executed and a StatCode of PXE_STATCODE_INVALID_CDB will be returned. The contents of the CPB structure will not be modified.</p>                                                         |
| DBsize  | <p><b>Data Block Siz</b></p> <p>This field should be set to a number that is equal to the number of bytes that will be written into the DB structure during command execution. Setting this field to a number that is smaller than required will cause an error. It may be zero in some cases where the information is not needed.</p>                                                                                                                             |
| CPBaddr | <p><b>Command Parameter Block Address</b></p> <p>For H/W UNDI, this field must be the physical address of the CPB structure. For S/W UNDI, this field must be the virtual address of the CPB structure. If the operation does not have/use a CPB, this field must be initialized to PXE_CPBADDR_NOT_USED. Setting up this field incorrectly will cause command execution to fail and a StatCode of PXE_STATCODE_INVALID_CDB will be returned.</p>                  |
| DBaddr  | <p><b>Data Block Address</b></p> <p>For H/W UNDI, this field must be the physical address of the DB structure. For S/W UNDI, this field must be the virtual address of the DB structure. If the operation does not have/use a CPB, this field must be initialized to PXE_DBADDR_NOT_USED. Setting up this field incorrectly will cause command execution to fail and a StatCode of PXE_STATCODE_INVALID_CDB will be returned.</p>                                  |

continues on next page

Table E.5 – continued from previous page

|           |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|-----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| StatCode  | <p>Status Code</p> <p>This field is used to report the type of command completion: success or failure (and the type of failure). This field must be initialized to zero before the command is issued. The contents of this field is not valid until the PXE_STATFLAGS_COMMAND_COMPLETE status flag is set. If this field is not initialized to PXE_STATCODE_INITIALIZE the UNDI command will not execute and a StatCode of PXE_STATCODE_INVALID_CDB will be returned.</p>                                                                                        |
| StatFlags | <p>Status Flags</p> <p>This bit field is used to report command completion and identify the format, if any, of the DB structure. This field must be initialized to zero before the command is issued. Until the command state changes to error or complete, all other CDB fields must not be changed. If this field is not initialized to PXE_STATFLAGS_INITIALIZE the UNDI command will not execute and a StatCode of PXE_STATCODE_INVALID_CDB will be returned. Bits 0x0F &amp; 0x0E: Command state: Not started (0), Queued (1), Error (2), Complete (3).</p> |
| IFnum     | <p>Interface Number</p> <p>This field is used to identify which network adapter (S/W UNDI) or network connector (H/W UNDI) this command is being sent to. If an invalid interface number is given, the command will not execute and a StatCode of PXE_STATCODE_INVALID_CDB will be returned.</p>                                                                                                                                                                                                                                                                 |
| Control   | <p>Process Control</p> <p>This bit field is used to control command UNDI inter-command processing. Setting control bits that are not supported by the UNDI will cause the command execution to fail with a StatCode of PXE_STATCODE_INVALID_CDB. Bit 0x00: Another CDB follows this one (1) or this is the last or only CDB in the list (0). Bit 0x01: Queue command if busy (1), fail if busy (0).</p>                                                                                                                                                          |

## E.3 UNDI C Definitions

The definitions in this section are used to aid in the portability and readability of the example 32/64-bit S/W UNDI source code and the rest of this specification.

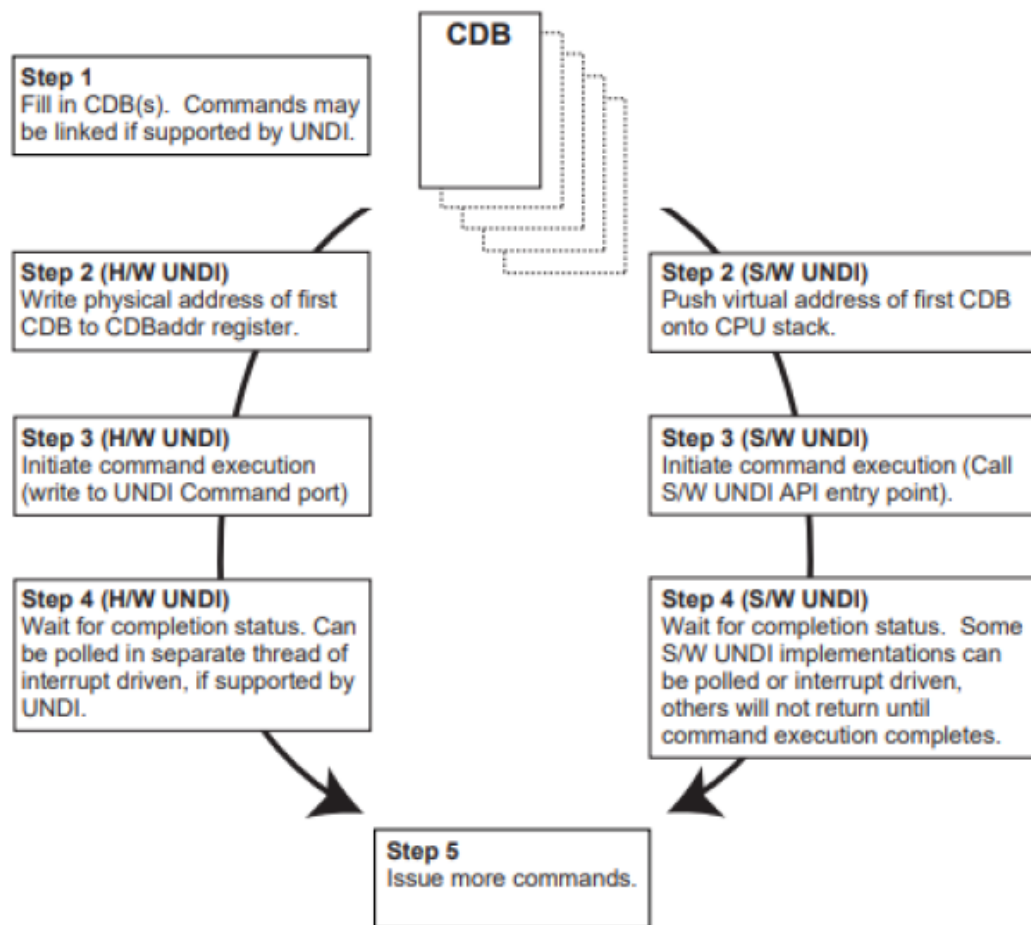


Fig. E.3: Issuing UNDI Commands

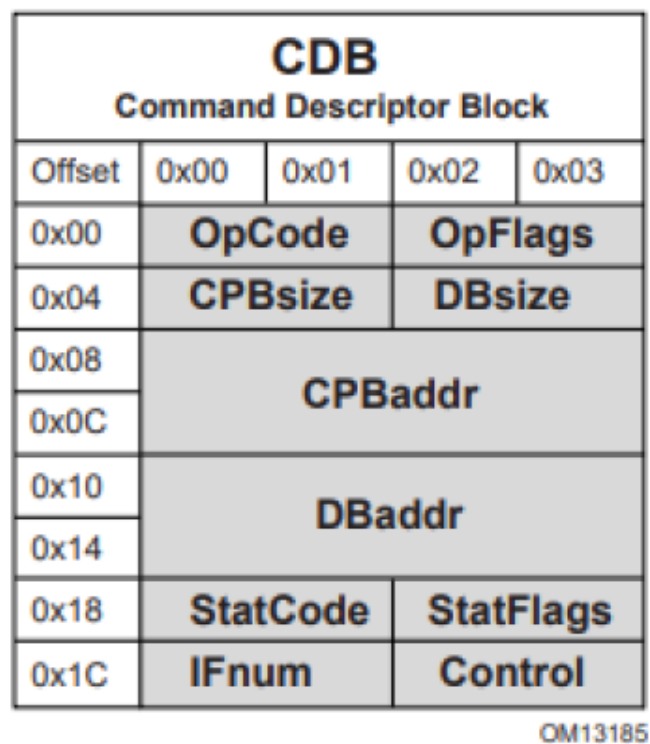


Fig. E.4: UNDI Command Descriptor Block (CDB)

E.3.1 Portability Macros

These macros are used for storage and communication portability.

E.3.1.1 PXE\_INTEL\_ORDER or PXE\_NETWORK\_ORDER

This macro is used to control conditional compilation in the S/W UNDI source code. One of these definitions needs to be uncommented in a common PXE header file.

```
// #define PXE_INTEL_ORDER 1 // little-endian
// #define PXE_NETWORK_ORDER 1 // big-endian
```

E.3.1.2 PXE\_UINT64\_SUPPORT or PXE\_NO\_UINT64\_SUPPORT

This macro is used to control conditional compilation in the PXE source code. One of these definitions must to be uncommented in the common PXE header file.

```
// #define PXE_UINT64_SUPPORT 1 // UINT64 supported
// #define PXE_NO_UINT64_SUPPORT 1 // UINT64 not supported
```

E.3.1.3 PXE\_BUSTYPE

Used to convert a 4-character ASCII identifier to a 32-bit unsigned integer.

```
#if PXE_INTEL_ORDER
#define PXE_BUSTYPE(a,b,c,d) \
(((PXE_UINT32)(d) & 0xFF) << 24) | \
(((PXE_UINT32)(c) & 0xFF) << 16) | \
(((PXE_UINT32)(b) & 0xFF) << 8) | \
((PXE_UINT32)(a) & 0xFF)

```

E.3. UNDI C Definitions2002



### E.3.1.4 PXE\_SWAP\_UINT16

This macro swaps bytes in a 16-bit word.

```
#ifndef PXE_INTEL_ORDER
#define PXE_SWAP_UINT16(n) \
(((PXE_UINT16)(n) & 0x00FF) << 8) | \
(((PXE_UINT16)(n) & 0xFF00) >> 8))
#else
#define PXE_SWAP_UINT16(n) (n)
#endif
```

### E.3.1.5 PXE\_SWAP\_UINT32

This macro swaps bytes in a 32-bit word.

```
#ifndef PXE_INTEL_ORDER
#define PXE_SWAP_UINT32(n) \
((((PXE_UINT32)(n) & 0x000000FF) << 24) | \
(((PXE_UINT32)(n) & 0x0000FF00) << 8) | \
(((PXE_UINT32)(n) & 0x00FF0000) >> 8) | \
(((PXE_UINT32)(n) & 0xFF000000) >> 24))
#else
#define PXE_SWAP_UINT32(n) (n)
#endif
```

### E.3.1.6 PXE\_SWAP\_UINT64

This macro swaps bytes in a 64-bit word for compilers that support 64-bit words.

```
#if PXE_UINT64_SUPPORT != 0
#ifndef PXE_INTEL_ORDER
#define PXE_SWAP_UINT64(n) \
((((((PXE_UINT64)(n) & 0x00000000000000FF) << 56) | \
(((PXE_UINT64)(n) & 0x000000000000FF00) << 40) | \
(((PXE_UINT64)(n) & 0x0000000000FF0000) << 24) | \
(((PXE_UINT64)(n) & 0x00000000FF000000) << 8) | \
(((PXE_UINT64)(n) & 0x000000FF00000000) >> 8) | \
(((PXE_UINT64)(n) & 0x0000FF0000000000) >> 24) | \
(((PXE_UINT64)(n) & 0x00FF000000000000) >> 40) | \
(((PXE_UINT64)(n) & 0xFF00000000000000) >> 56))
#else
#define PXE_SWAP_UINT64(n) (n)
#endif
#endif // PXE_UINT64_SUPPORT
```

This macro swaps bytes in a 64-bit word, in place, for compilers that do not support 64-bit words. This version of the 64-bit swap macro cannot be used in expressions.

```
#if PXE_NO_UINT64_SUPPORT != 0
#if PXE_INTEL_ORDER
#define PXE_SWAP_UINT64(n) \
```

(continues on next page)

(continued from previous page)

```

{
    \
    PXE_UINT32 tmp = (PXE_UINT64)(n)[1]; \
    (PXE_UINT64)(n)[1] = PXE_SWAP_UINT32((PXE_UINT64)(n)[0]); \
    (PXE_UINT64)(n)[0] = PXE_SWAP_UINT32(tmp); \
}
#else
#define PXE_SWAP_UINT64(n) (n)
#endif
#endif // PXE_NO_UINT64_SUPPORT

```

## E.3.2 Miscellaneous Macros

### E.3.2.1 Miscellaneous

```

#define PXE_CPBSIZE_NOT_USED 0 // zero
#define PXE_DBSIZE_NOT_USED 0 // zero
#define PXE_CPBADDR_NOT_USED (PXE_UINT64)0 // zero
#define PXE_DBADDR_NOT_USED (PXE_UINT64)0 // zero

```

## E.3.3 Portability Types

The examples given below are just that, examples. The actual typedef instructions used in a new implementation may vary depending on the compiler and processor architecture.

The storage sizes defined in this section are critical for PXE module inter-operation. All of the portability typedefs define little endian (Intel® format) storage. The least significant byte is stored in the lowest memory address and the most significant byte is stored in the highest memory address, as shown in See *Storage Types*.

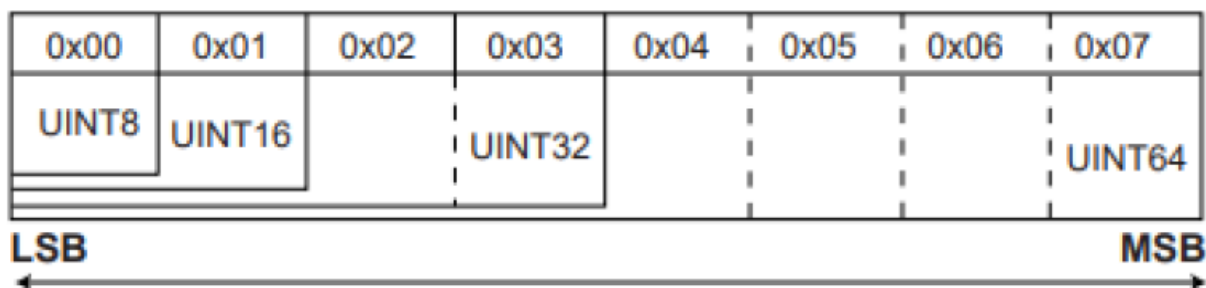


Fig. E.5: Storage Types

### E.3.3.1 PXE\_CONST

The const type does not allocate storage. This type is a modifier that is used to help the compiler optimize parameters that do not change across function calls.

```
#define PXE_CONST const
```

### E.3.3.2 PXE\_VOLATILE

The volatile type does not allocate storage. This type is a modifier that is used to help the compiler deal with variables that can be changed by external procedures or hardware events.

```
#define PXE_VOLATILE volatile
```

### E.3.3.3 PXE\_VOID

The void type does not allocate storage. This type is used only to prototype functions that do not return any information and/or do not take any parameters.

```
typedef void PXE_VOID;
```

### E.3.3.4 PXE\_UINT8

Unsigned 8-bit integer.

```
typedef unsigned char PXE_UINT8;
```

### E.3.3.5 PXE\_UINT16

Unsigned 16-bit integer.

```
typedef unsigned short PXE_UINT16;
```

### E.3.3.6 PXE\_UINT32

Unsigned 32-bit integer.

```
typedef unsigned PXE_UINT32;
```

### E.3.3.7 PXE\_UINT64

Unsigned 64-bit integer.

```
#if PXE_UINT64_SUPPORT != 0
typedef unsigned long PXE_UINT64;
#endif // PXE_UINT64_SUPPORT
```

If a 64-bit integer type is not available in the compiler being used, use this definition:

```
#if PXE_NO_UINT64_SUPPORT != 0
typedef PXE_UINT32 PXE_UINT64[2];
#endif // PXE_NO_UINT64_SUPPORT
```

### E.3.3.8 PXE\_UINTN

Unsigned integer that is the default word size used by the compiler. This needs to be at least a 32-bit unsigned integer.

```
typedef unsigned PXE_UINTN;
```

## E.3.4 Simple Types

The PXE simple types are defined using one of the portability types from the previous section.

### E.3.4.1 PXE\_BOOL

Boolean (true/false) data type. For PXE zero is always false and nonzero is always true.

```
typedef PXE_UINT8 PXE_BOOL;
#define PXE_FALSE 0 // zero
#define PXE_TRUE (!PXE_FALSE)
```

### E.3.4.2 PXE\_OPCODE

UNDI OpCode (command) descriptions are given in the next chapter. There are no BC OpCodes, BC protocol functions are discussed later in this document.

```
typedef PXE_UINT16 PXE_OPCODE;

// Return UNDI operational state.
#define PXE_OPCODE_GET_STATE 0x0000

// Change UNDI operational state from Stopped to Started.
#define PXE_OPCODE_START 0x0001

// Change UNDI operational state from Started to Stopped.
#define PXE_OPCODE_STOP 0x0002

// Get UNDI initialization information.
#define PXE_OPCODE_GET_INIT_INFO 0x0003
```

(continues on next page)

(continued from previous page)

```
// Get NIC configuration information.
#define PXE_OPCODE_GET_CONFIG_INFO 0x0004

// Changed UNDI operational state from Started to Initialized.
#define PXE_OPCODE_INITIALIZE 0x0005

// Reinitialize the NIC H/W.
#define PXE_OPCODE_RESET 0x0006

// Change the UNDI operational state from Initialized to Started.
#define PXE_OPCODE_SHUTDOWN 0x0007

// Read & change state of external interrupt enables.
#define PXE_OPCODE_INTERRUPT_ENABLES 0x0008

// Read & change state of packet receive filters.
#define PXE_OPCODE_RECEIVE_FILTERS 0x0009

// Read & change station MAC address.
#define PXE_OPCODE_STATION_ADDRESS 0x000A

// Read traffic statistics.
#define PXE_OPCODE_STATISTICS 0x000B

// Convert multicast IP address to multicast MAC address.
#define PXE_OPCODE_MCAST_IP_TO_MAC 0x000C

// Read or change nonvolatile storage on the NIC.
#define PXE_OPCODE_NVDATA 0x000D

// Get & clear interrupt status.
#define PXE_OPCODE_GET_STATUS 0x000E

// Fill media header in packet for transmit.
#define PXE_OPCODE_FILL_HEADER 0x000F

// Transmit packet(s).
#define PXE_OPCODE_TRANSMIT 0x0010

// Receive packet.
#define PXE_OPCODE_RECEIVE 0x0011

// Last valid PXE UNDI OpCode number.
#define PXE_OPCODE_LAST_VALID 0x0011
```

**E.3.4.3 PXE\_OPFLAGS**

```

typedef PXE_UINT16 PXE_OPFLAGS;

#define PXE_OPFLAGS_NOT_USED 0x0000

//*****
// UNDI Get State
//*****

// No OpFlags

//*****
// UNDI Start
//*****

// No OpFlags

//*****
// UNDI Stop
//*****

// No OpFlags

//*****
// UNDI Get Init Info
//*****

// No Opflags

//*****
// UNDI Get Config Info
//*****

// No Opflags

//*****
// UNDI Initialize
//*****

#define PXE_OPFLAGS_INITIALIZE_CABLE_DETECT_MASK 0x0001
#define PXE_OPFLAGS_INITIALIZE_DETECT_CABLE 0x0000
#define PXE_OPFLAGS_INITIALIZE_DO_NOT_DETECT_CABLE 0x0001

//*****
// UNDI Reset
//*****

#define PXE_OPFLAGS_RESET_DISABLE_INTERRUPTS 0x0001
#define PXE_OPFLAGS_RESET_DISABLE_FILTERS 0x0002

//*****
// UNDI Shutdown

```

(continues on next page)

(continued from previous page)

```

//*****

// No OpFlags

//*****
// UNDI Interrupt Enables
//*****

// Select whether to enable or disable external interrupt
// signals. Setting both enable and disable will return
// PXE_STATCODE_INVALID_OPFLAGS.

#define PXE_OPFLAGS_INTERRUPT_OPMASK 0xC000
#define PXE_OPFLAGS_INTERRUPT_ENABLE 0x8000
#define PXE_OPFLAGS_INTERRUPT_DISABLE 0x4000
#define PXE_OPFLAGS_INTERRUPT_READ 0x0000

// Enable receive interrupts. An external interrupt will be
// generated after a complete non-error packet has been received.

#define PXE_OPFLAGS_INTERRUPT_RECEIVE 0x0001

// Enable transmit interrupts. An external interrupt will be
// generated after a complete non-error packet has been
// transmitted.

#define PXE_OPFLAGS_INTERRUPT_TRANSMIT 0x0002

// Enable command interrupts. An external interrupt will be
// generated when command execution stops.

#define PXE_OPFLAGS_INTERRUPT_COMMAND 0x0004

// Generate software interrupt. Setting this bit generates an
// external interrupt, if it is supported by the hardware.

#define PXE_OPFLAGS_INTERRUPT_SOFTWARE 0x0008

//*****
// UNDI Receive Filters
//*****

// Select whether to enable or disable receive filters.
// Setting both enable and disable will return
// PXE_STATCODE_INVALID_OPCODE.

#define PXE_OPFLAGS_RECEIVE_FILTER_OPMASK 0xC000
#define PXE_OPFLAGS_RECEIVE_FILTER_ENABLE 0x8000
#define PXE_OPFLAGS_RECEIVE_FILTER_DISABLE 0x4000
#define PXE_OPFLAGS_RECEIVE_FILTER_READ 0x0000

```

(continues on next page)

(continued from previous page)

```

// To reset the contents of the multicast MAC address filter
// list, set this OpFlag:

#define PXE_OPFLAGS_RECEIVE_FILTERS_RESET_MCAST_LIST 0x2000

// Enable unicast packet receiving. Packets sent to the
// current station MAC address will be received.

#define PXE_OPFLAGS_RECEIVE_FILTER_UNICAST 0x0001

// Enable broadcast packet receiving. Packets sent to the
// broadcast MAC address will be received.

#define PXE_OPFLAGS_RECEIVE_FILTER_BROADCAST 0x0002

// Enable filtered multicast packet receiving. Packets sent to
// any of the multicast MAC addresses in the multicast MAC
// address filter list will be received. If the filter list is
// empty, no multicast

#define PXE_OPFLAGS_RECEIVE_FILTER_FILTERED_MULTICAST 0x0004

// Enable promiscuous packet receiving. All packets will be
// received.

#define PXE_OPFLAGS_RECEIVE_FILTER_PROMISCUOUS 0x0008

// Enable promiscuous multicast packet receiving. All multicast
// packets will be received.

#define PXE_OPFLAGS_RECEIVE_FILTER_ALL_MULTICAST 0x0010

//*****
// UNDI Station Address
//*****

#define PXE_OPFLAGS_STATION_ADDRESS_READ 0x0000
#define PXE_OPFLAGS_STATION_ADDRESS_WRITE 0x0000
#define PXE_OPFLAGS_STATION_ADDRESS_RESET 0x0001

//*****
// UNDI Statistics
//*****

#define PXE_OPFLAGS_STATISTICS_READ 0x0000
#define PXE_OPFLAGS_STATISTICS_RESET 0x0001

//*****
// UNDI MCast IP to MAC
//*****

// Identify the type of IP address in the CPB.

```

(continues on next page)



(continued from previous page)

```

#define PXE_OPFLAGS_MCAST_IP_TO_MAC_OPMASK 0x0003
#define PXE_OPFLAGS_MCAST_IPV4_TO_MAC 0x0000
#define PXE_OPFLAGS_MCAST_IPV6_TO_MAC 0x0001

//*****
// UNDI NvData
//*****

// Select the type of nonvolatile data operation.

#define PXE_OPFLAGS_NVDATA_OPMASK 0x0001
#define PXE_OPFLAGS_NVDATA_READ 0x0000
#define PXE_OPFLAGS_NVDATA_WRITE 0x0001

//*****
// UNDI Get Status
//*****

// Return current interrupt status. This will also clear any
// interrupts that are currently set. This can be used in a
// polling routine. The interrupt flags are still set and
// cleared even when the interrupts are disabled.

#define PXE_OPFLAGS_GET_INTERRUPT_STATUS 0x0001

// Return list of transmitted buffers for recycling. Transmit
// buffers must not be changed or unallocated until they have
// recycled. After issuing a transmit command, wait for a
// transmit complete interrupt. When a transmit complete
// interrupt is received, read the transmitted buffers. Do not
// plan on getting one buffer per interrupt. Some NICs and UNDIIs
// may transmit multiple buffers per interrupt.

#define PXE_OPFLAGS_GET_TRANSMITTED_BUFFERS 0x0002

// Return current media status.

#define PXE_OPFLAGS_GET_MEDIA_STATUS 0x0004

//*****
// UNDI Fill Header
//*****

#define PXE_OPFLAGS_FILL_HEADER_OPMASK 0x0001
#define PXE_OPFLAGS_FILL_HEADER_FRAGMENTED 0x0001
#define PXE_OPFLAGS_FILL_HEADER_WHOLE 0x0000

//*****
// UNDI Transmit
//*****

```

(continues on next page)

(continued from previous page)

```
// S/W UNDI only. Return after the packet has been transmitted.
// A transmit complete interrupt will still be generated and the
// transmit buffer will have to be recycled.

#define PXE_OPFLAGS_SWUNDI_TRANSMIT_OPMASK 0x0001
#define PXE_OPFLAGS_TRANSMIT_BLOCK 0x0001
#define PXE_OPFLAGS_TRANSMIT_DONT_BLOCK 0x0000

#define PXE_OPFLAGS_TRANSMIT_OPMASK 0x0002
#define PXE_OPFLAGS_TRANSMIT_FRAGMENTED 0x0002
#define PXE_OPFLAGS_TRANSMIT_WHOLE 0x0000

//*****
// UNDI Receive
//*****

// No OpFlags
```

#### E.3.4.4 PXE\_STATFLAGS

```
typedef PXE_UINT16 PXE_STATFLAGS;

#define PXE_STATFLAGS_INITIALIZE 0x0000

//*****
// Common StatFlags that can be returned by all commands.
//*****

// The COMMAND_COMPLETE and COMMAND_FAILED status flags must be
// implemented by all UNDIIs. COMMAND_QUEUED is only needed by
// UNDIIs that support command queuing.

#define PXE_STATFLAGS_STATUS_MASK 0xC000
#define PXE_STATFLAGS_COMMAND_COMPLETE 0xC000
#define PXE_STATFLAGS_COMMAND_FAILED 0x8000
#define PXE_STATFLAGS_COMMAND_QUEUED 0x4000

//*****
// UNDI Get State
//*****

#define PXE_STATFLAGS_GET_STATE_MASK 0x0003
#define PXE_STATFLAGS_GET_STATE_INITIALIZED 0x0002
#define PXE_STATFLAGS_GET_STATE_STARTED 0x0001
#define PXE_STATFLAGS_GET_STATE_STOPPED 0x0000

//*****
// UNDI Start
//*****
```

(continues on next page)

(continued from previous page)

```
// No additional StatFlags

//*****
// UNDI Get Init Info
//*****

#define PXE_STATFLAGS_CABLE_DETECT_MASK 0x0001
#define PXE_STATFLAGS_CABLE_DETECT_NOT_SUPPORTED 0x0000
#define PXE_STATFLAGS_CABLE_DETECT_SUPPORTED 0x0001
#define PXE_STATFLAGS_GET_STATUS_NO_MEDIA_MASK 0x0002
#define PXE_STATFLAGS_GET_STATUS_NO_MEDIA_NOT_SUPPORTED 0x0000
#define PXE_STATFLAGS_GET_STATUS_NO_MEDIA_SUPPORTED 0x0002

//*****
// UNDI Initialize
//*****

#define PXE_STATFLAGS_INITIALIZED_NO_MEDIA 0x0001

//*****
// UNDI Reset
//*****

#define PXE_STATFLAGS_RESET_NO_MEDIA 0x0001

//*****
// UNDI Shutdown
//*****

// No additional StatFlags

//*****
// UNDI Interrupt Enables
//*****

// If set, receive interrupts are enabled.
#define PXE_STATFLAGS_INTERRUPT_RECEIVE 0x0001

// If set, transmit interrupts are enabled.
#define PXE_STATFLAGS_INTERRUPT_TRANSMIT 0x0002

// If set, command interrupts are enabled.
#define PXE_STATFLAGS_INTERRUPT_COMMAND 0x0004

//*****
// UNDI Receive Filters
//*****

// If set, unicast packets will be received.
#define PXE_STATFLAGS_RECEIVE_FILTER_UNICAST 0x0001

// If set, broadcast packets will be received.
```

(continues on next page)

(continued from previous page)

```
#define PXE_STATFLAGS_RECEIVE_FILTER_BROADCAST 0x0002

// If set, multicast packets that match up with the multicast
// address filter list will be received.
#define PXE_STATFLAGS_RECEIVE_FILTER_FILTERED_MULTICAST 0x0004

// If set, all packets will be received.
#define PXE_STATFLAGS_RECEIVE_FILTER_PROMISCUOUS 0x0008

// If set, all multicast packets will be received.
#define PXE_STATFLAGS_RECEIVE_FILTER_ALL_MULTICAST 0x0010

//*****
// UNDI Station Address
//*****

// No additional StatFlags

//*****
// UNDI Statistics
//*****

// No additional StatFlags

//*****
// UNDI MCast IP to MAC
//*****

// No additional StatFlags

//*****
// UNDI NvData
//*****

// No additional StatFlags

//*****
// UNDI Get Status
//*****

// Use to determine if an interrupt has occurred.
#define PXE_STATFLAGS_GET_STATUS_INTERRUPT_MASK 0x000F
#define PXE_STATFLAGS_GET_STATUS_NO_INTERRUPTS 0x0000

// If set, at least one receive interrupt occurred.
#define PXE_STATFLAGS_GET_STATUS_RECEIVE 0x0001

// If set, at least one transmit interrupt occurred.

#define PXE_STATFLAGS_GET_STATUS_TRANSMIT 0x0002

// If set, at least one command interrupt occurred.
```

(continues on next page)

(continued from previous page)

```

#define PXE_STATFLAGS_GET_STATUS_COMMAND 0x0004

// If set, at least one software interrupt occurred.
#define PXE_STATFLAGS_GET_STATUS_SOFTWARE 0x0008

// This flag is set if the transmitted buffer queue is empty.
// This flag will be set if all transmitted buffer addresses
// get written into the DB.
#define PXE_STATFLAGS_GET_STATUS_TXBUF_QUEUE_EMPTY 0x0010

// This flag is set if no transmitted buffer addresses were
// written into the DB. (This could be because DBsize was
// too small.)
#define PXE_STATFLAGS_GET_STATUS_NO_TXBUFS_WRITTEN 0x0020

// This flag is set if there is no media detected
#define PXE_STATFLAGS_GET_STATUS_NO_MEDIA 0x0040

//*****
// UNDI Fill Header
//*****

// No additional StatFlags

//*****
// UNDI Transmit
//*****

// No additional StatFlags.

//*****
// UNDI Receive
//*****

// No additional StatFlags.

```

### E.3.4.5 PXE\_STATCODE

```

typedef PXE_UINT16 PXE_STATCODE;

#define PXE_STATCODE_INITIALIZE 0x0000

//*****
// Common StatCodes returned by all UNDI commands, UNDI protocol
// functions and BC protocol functions.
//*****

#define PXE_STATCODE_SUCCESS 0x0000
#define PXE_STATCODE_INVALID_CDB 0x0001
#define PXE_STATCODE_INVALID_CPB 0x0002

```

(continues on next page)

(continued from previous page)

```

#define PXE_STATCODE_BUSY 0x0003
#define PXE_STATCODE_QUEUE_FULL 0x0004
#define PXE_STATCODE_ALREADY_STARTED 0x0005
#define PXE_STATCODE_NOT_STARTED 0x0006
#define PXE_STATCODE_NOT_SHUTDOWN 0x0007
#define PXE_STATCODE_ALREADY_INITIALIZED 0x0008
#define PXE_STATCODE_NOT_INITIALIZED 0x0009
#define PXE_STATCODE_DEVICE_FAILURE 0x000A
#define PXE_STATCODE_NVDATA_FAILURE 0x000B
#define PXE_STATCODE_UNSUPPORTED 0x000C
#define PXE_STATCODE_BUFFER_FULL 0x000D
#define PXE_STATCODE_INVALID_PARAMETER 0x000E
#define PXE_STATCODE_INVALID_UNDI 0x000F
#define PXE_STATCODE_IPV4_NOT_SUPPORTED 0x0010
#define PXE_STATCODE_IPV6_NOT_SUPPORTED 0x0011
#define PXE_STATCODE_NOT_ENOUGH_MEMORY 0x0012
#define PXE_STATCODE_NO_DATA 0x0013

```

#### E.3.4.6 PXE\_IFNUM

```

typedef PXE_UINT16 PXE_IFNUM;
// This interface number must be passed to the S/W UNDI Start
// command.

#define PXE_IFNUM_START 0x0000

// This interface number is returned by the S/W UNDI Get State
// and Start commands if information in the CDB, CPB or DB is
// invalid.

#define PXE_IFNUM_INVALID 0x0000

```

#### E.3.4.7 PXE\_CONTROL

```

typedef PXE_UINT16 PXE_CONTROL;

// Setting this flag directs the UNDI to queue this command for
// later execution if the UNDI is busy and it supports command
// queuing. If queuing is not supported, a
// PXE_STATCODE_INVALID_CONTROL error is returned. If the queue
// is full, a PXE_STATCODE_CDB_QUEUE_FULL error is returned.

#define PXE_CONTROL_QUEUE_IF_BUSY 0x0002

// These two bit values are used to determine if there are more
// UNDI CDB structures following this one. If the link bit is
// set, there must be a CDB structure following this one.
// Execution will start on the next CDB structure as soon as this
// one completes successfully. If an error is generated by this

```

(continues on next page)

(continued from previous page)

```
// command, execution will stop.

#define PXE_CONTROL_LINK 0x0001
#define PXE_CONTROL_LAST_CDB_IN_LIST 0x0000
```

#### E.3.4.8 PXE\_FRAME\_TYPE

```
typedef PXE_UINT8 PXE_FRAME_TYPE;

#define PXE_FRAME_TYPE_NONE 0x00
#define PXE_FRAME_TYPE_UNICAST 0x01
#define PXE_FRAME_TYPE_BROADCAST 0x02
#define PXE_FRAME_TYPE_FILTERED_MULTICAST 0x03
#define PXE_FRAME_TYPE_PROMISCUOUS 0x04
#define PXE_FRAME_TYPE_PROMISCUOUS_MULTICAST 0x05
```

#### E.3.4.9 PXE\_IPV4

This storage type is always big endian, not little endian.

```
typedef PXE_UINT32 PXE_IPV4;
```

#### E.3.4.10 PXE\_IPV6

This storage type is always big endian, not little endian.

```
typedef struct s_PXE_IPV6 {
    PXE_UINT32 *num* [4];
} PXE_IPV6;
```

#### E.3.4.11 PXE\_MAC\_ADDR

This storage type is always big endian, not little endian.

```
typedef struct {
    PXE_UINT8 *num* [32];
} PXE_MAC_ADDR;
```

### E.3.4.12 PXE\_IFTYPE

The interface type is returned by the Get Initialization Information command and is used by the BC DHCP protocol function. This field is also used for the low order 8-bits of the H/W type field in ARP packets. The high order 8-bits of the H/W type field in ARP packets will always be set to 0x00 by the BC.

```
typedef PXE_UINT8 PXE_IFTYPE;

// This information is from the ARP section of RFC 3232.

// 1 Ethernet (10Mb)
// 2 Experimental Ethernet (3Mb)
// 3 Amateur Radio AX.25
// 4 Proteon ProNET Token Ring
// 5 Chaos
// 6 IEEE 802 Networks
// 7 ARCNET
// 8 Hyperchannel
// 9 Lanstar
// 10 Autonet Short Address
// 11 LocalTalk
// 12 LocalNet (IBM PCNet or SYTEK LocalNET)
// 13 Ultra link
// 14 SMDS
// 15 Frame Relay
// 16 Asynchronous Transmission Mode (ATM)
// 17 HDLC
// 18 Fibre Channel
// 19 Asynchronous Transmission Mode (ATM)
// 20 Serial Line
// 21 Asynchronous Transmission Mode (ATM)

#define PXE_IFTYPE_ETHERNET 0x01
#define PXE_IFTYPE_TOKENRING 0x04
#define PXE_IFTYPE_FIBRE_CHANNEL 0x12
```

### E.3.4.13 PXE\_MEDIA\_PROTOCOL

Protocol type. This will be copied into the media header without doing byte swapping. Protocol type numbers can be obtained from the assigned numbers RFC 3232.

```
typedef UINT16 PXE_MEDIA_PROTOCOL;
```



## E.3.5 Compound Types

All PXE structures must be byte packed.

### E.3.5.1 PXE\_HW\_UNDI

This section defines the C structures and #defines for the !PXE H/W UNDI interface.

```
#pragma pack(1)
typedef struct s_pxe_hw_undi {
    PXE_UINT32    Signature; // PXE_ROMID_SIGNATURE
    PXE_UINT8     Len; // sizeof(PXE_HW_UNDI)
    PXE_UINT8     Fudge; // makes 8-bit cksum equal zero
    PXE_UINT8     Rev; // PXE_ROMID_REV
    PXE_UINT8     IFcnt; // physical connector count
    lower byte
    PXE_UINT8     MajorVer; // PXE_ROMID_MAJORVER
    PXE_UINT8     MinorVer; // PXE_ROMID_MINORVER
    PXE_UINT8     IFcntExt; // physical connector count
    upper byte
    PXE_UINT8     reserved; // zero, not used
    PXE_UINT32     Implementation; // implementation flags
}    PXE_HW_UNDI;
#pragma pack()

// Status port bit definitions

// UNDI operation state

#define PXE_HWSTAT_STATE_MASK    0xC0000000
#define PXE_HWSTAT_BUSY    0xC0000000
#define PXE_HWSTAT_INITIALIZED    0x80000000
#define PXE_HWSTAT_STARTED    0x40000000
#define PXE_HWSTAT_STOPPED    0x00000000

// If set, last command failed

#define PXE_HWSTAT_COMMAND_FAILED    0x20000000

// If set, identifies enabled receive

filters <http://www.ietf.org/rfc/rfc1700.txt>__`
#define PXE_HWSTAT_PROMISCUOUS_MULTICAST_RX_ENABLED    0x00001000
#define PXE_HWSTAT_PROMISCUOUS_RX_ENABLED    0x00000800
#define PXE_HWSTAT_BROADCAST_RX_ENABLED    0x00000400
#define PXE_HWSTAT_MULTICAST_RX_ENABLED    0x00000200
#define PXE_HWSTAT_UNICAST_RX_ENABLED    0x00000100

// If set, identifies enabled external interrupts

#define PXE_HWSTAT_SOFTWARE_INT_ENABLED    0x00000080
#define PXE_HWSTAT_TX_COMPLETE_INT_ENABLED    0x00000040
#define PXE_HWSTAT_PACKET_RX_INT_ENABLED    0x00000020
```

(continues on next page)

(continued from previous page)

```
#define PXE_HWSTAT_CMD_COMPLETE_INT_ENABLED 0x00000010

// If set, identifies pending interrupts

#define PXE_HWSTAT_SOFTWARE_INT_PENDING 0x00000008
#define PXE_HWSTAT_TX_COMPLETE_INT_PENDING 0x00000004
#define PXE_HWSTAT_PACKET_RX_INT_PENDING 0x00000002
#define PXE_HWSTAT_CMD_COMPLETE_INT_PENDING 0x00000001

// Command port definitions

// If set, CDB identified in CDBaddr port is given to UNDI
// If not set, other bits in this word will be processed.

#define PXE_HWCMD_ISSUE_COMMAND 0x80000000
#define PXE_HWCMD_INTS_AND_FILTERS 0x00000000

// Use these to enable/disable receive filters.

#define PXE_HWCMD_PROMISCUOUS_MULTICAST_RX_ENABLE 0x00001000
#define PXE_HWCMD_PROMISCUOUS_RX_ENABLE 0x00000800
#define PXE_HWCMD_BROADCAST_RX_ENABLE 0x00000400
#define PXE_HWCMD_MULTICAST_RX_ENABLE 0x00000200
#define PXE_HWCMD_UNICAST_RX_ENABLE 0x00000100

// Use these to enable/disable external interrupts

#define PXE_HWCMD_SOFTWARE_INT_ENABLE 0x00000080
#define PXE_HWCMD_TX_COMPLETE_INT_ENABLE 0x00000040
#define PXE_HWCMD_PACKET_RX_INT_ENABLE 0x00000020
#define PXE_HWCMD_CMD_COMPLETE_INT_ENABLE 0x00000010

// Use these to clear pending external interrupts

#define PXE_HWCMD_CLEAR_SOFTWARE_INT 0x00000008
#define PXE_HWCMD_CLEAR_TX_COMPLETE_INT 0x00000004
#define PXE_HWCMD_CLEAR_PACKET_RX_INT 0x00000002
#define PXE_HWCMD_CLEAR_CMD_COMPLETE_INT 0x00000001
```

### E.3.5.2 PXE\_SW\_UNDI

This section defines the C structures and #defines for the !PXE S/W UNDI interface.

```
#pragma pack(1)
typedef struct s_pxe_sw_undi {
    PXE_UINT32    Signature // PXE_ROMID_SIGNATURE
    PXE_UINT8     Len; // sizeof(PXE_SW_UNDI)
    PXE_UINT8     Fudge; // makes 8-bit cksum zero
    PXE_UINT8     Rev; // PXE_ROMID_REV
    PXE_UINT8     Fcnt; // physical connector count
    lower byte
```

(continues on next page)

(continued from previous page)

```

PXE_UINT8    MajorVer // PXE_ROMID_MAJORVER
PXE_UINT8    MinorVer; // PXE_ROMID_MINORVER
PXE_UINT8    IFcntExt; // physical connector count
                upper byte
PXE_UINT8    reserved1; // zero, not used
PXE_UINT32    Implementation; // Implementation flags
PXE_UINT64    EntryPoint; // API entry point
PXE_UINT8    reserved2[3]; // zero, not used
PXE_UINT8    BusCnt; // number of bustypes supported
PXE_UINT32    BusType[1]; // list of supported bustypes
} PXE_SW_UNDI;
#pragma pack()

```

### E.3.5.3 PXE\_UNDI

PXE\_UNDI combines both the H/W and S/W UNDI types into one typedef and has #defines for common fields in both H/W and S/W UNDI types.

```

#pragma pack(1)
typedef union u_pxe_undi {
    PXE_HW_UNDI hw;
    PXE_SW_UNDI sw;
} PXE_UNDI;
#pragma pack()

// Signature of !PXE structure

#define PXE_ROMID_SIGNATURE PXE_BUSTYPE ('!', 'P', 'X', 'E')

// !PXE structure format revision *)*
// See "Links to UEFI-Related Documents" (http://uefi.org/uefi)
// under the heading "UDP over IPv6".

#define PXE_ROMID_REV 0x02

// UNDI command interface revision. These are the values that
// get sent in option 94 (Client Network Interface Identifier) in
// the DHCP Discover and PXE Boot Server Request packets.
// See "Links to UEFI-Related Documents" (http://uefi.org/uefi)
// under the heading "IETF Organization".

#define PXE_ROMID_MAJORVER 0x03
#define PXE_ROMID_MINORVER 0x01

// Implementation flags

#define PXE_ROMID_IMP_HW_UNDI 0x80000000
#define PXE_ROMID_IMP_SW_VIRT_ADDR 0x40000000
#define PXE_ROMID_IMP_64BIT_DEVICE 0x00010000
#define PXE_ROMID_IMP_FRAG_SUPPORTED 0x00008000
#define PXE_ROMID_IMP_CMD_LINK_SUPPORTED 0x00004000

```

(continues on next page)

(continued from previous page)

```

#define PXE_ROMID_IMP_CMD_QUEUE_SUPPORTED 0x00002000
#define PXE_ROMID_IMP_MULTI_FRAME_SUPPORTED 0x00001000
#define PXE_ROMID_IMP_NVDATA_SUPPORT_MASK 0x00000C00
#define PXE_ROMID_IMP_NVDATA_BULK_WRITABLE 0x00000C00
#define PXE_ROMID_IMP_NVDATA_SPARSE_WRITABLE 0x00000800
#define PXE_ROMID_IMP_NVDATA_READ_ONLY 0x00000400
#define PXE_ROMID_IMP_NVDATA_NOT_AVAILABLE 0x00000000
#define PXE_ROMID_IMP_STATISTICS_SUPPORTED 0x00000200
#define PXE_ROMID_IMP_STATION_ADDR_SETTABLE 0x00000100
#define PXE_ROMID_IMP_PROMISCUOUS_MULTICAST_RX_SUPPORTED 0x00000080
#define PXE_ROMID_IMP_PROMISCUOUS_RX_SUPPORTED 0x00000040
#define PXE_ROMID_IMP_BROADCAST_RX_SUPPORTED 0x00000020
#define PXE_ROMID_IMP_FILTERED_MULTICAST_RX_SUPPORTED 0x00000010
#define PXE_ROMID_IMP_SOFTWARE_INT_SUPPORTED 0x00000008
#define PXE_ROMID_IMP_TX_COMPLETE_INT_SUPPORTED 0x00000004
#define PXE_ROMID_IMP_PACKET_RX_INT_SUPPORTED 0x00000002
#define PXE_ROMID_IMP_CMD_COMPLETE_INT_SUPPORTED 0x00000001

```

### E.3.5.4 PXE\_CDB

PXE UNDI command descriptor block.

```

#pragma pack(1)
typedef struct s_pxe_cdb {
    PXE_OPCODE      OpCode;
    PXE_OPFLAGS     OpFlags;
    PXE_UINT16      CPBsize;
    PXE_UINT16      DBsize;
    PXE_UINT64      CPBaddr;
    PXE_UINT64      DBaddr;
    PXE_STATCODE     StatCode;
    PXE_STATFLAGS    StatFlags;
    PXE_UINT16      IFnum;
    PXE_CONTROL      Control;
} PXE_CDB;
#pragma pack()

```

If the UNDI driver enables hardware VLAN support, UNDI driver could use *IFnum* to identify the real NICs and VLAN created virtual NICs.

### E.3.5.5 PXE\_IP\_ADDR

This storage type is always big endian, not little endian.

```

#pragma pack(1)
typedef union u_pxe_ip_addr {
    PXE_IPV6      IPv6;
    PXE_IPV4      IPv4;
} PXE_IP_ADDR;
#pragma pack()

```

### E.3.5.6 PXE\_DEVICE

This typedef is used to identify the network device that is being used by the UNDI. This information is returned by the Get Config Info command.

```
#pragma pack(1)
typedef union pxe_device {

    // PCI and PC Card NICs are both identified using bus, device
    // and function numbers. For PC Card, this may require PC
    // Card services to be loaded in the BIOS or preboot
    // environment.
    struct {
        // See S/W UNDI ROMID structure definition for PCI and
        // PCC BusType definitions.
        PXE_UINT32      BusType;

        // Bus, device & function numbers that locate this device.
        PXE_UINT16      Bus;
        PXE_UINT8       Device;
        PXE_UINT8       Function;
    }    PCI, PCC;

}    PXE_DEVICE;
#pragma pack()
```

## E.4 UNDI Commands

All 32/64-bit UNDI commands use the same basic command format, the CDB (Command Descriptor Block). CDB fields that are not used by a particular command must be initialized to zero by the application/driver that is issuing the command. (See “Links to UEFI-Related Documents” (<http://uefi.org/uefi>) under the heading “DMTF BIOS specifications”).

All UNDI implementations must set the command completion status ( *PXE\_STATFLAGS\_COMMAND\_COMPLETE* ) after command execution completes. Applications and drivers must not alter or rely on the contents of any of the CDB, CPB or DB fields until the command completion status is set.

All commands return status codes for invalid CDB contents and, if used, invalid CPB contents. Commands with invalid parameters will not execute. Fix the error and submit the command again.

See the Figure, below, *UNDI States, Transitions & Valid Commands* describes the different UNDI states (Stopped, Started and Initialized), shows the transitions between the states and which UNDI commands are valid in each state.

#### Note

All memory addresses including the CDB address, CPB address, and the DB address submitted to the S/W UNDI by the protocol drivers must be processor-based addresses. All memory addresses submitted to the H/W UNDI must be device based addresses.

#### Note

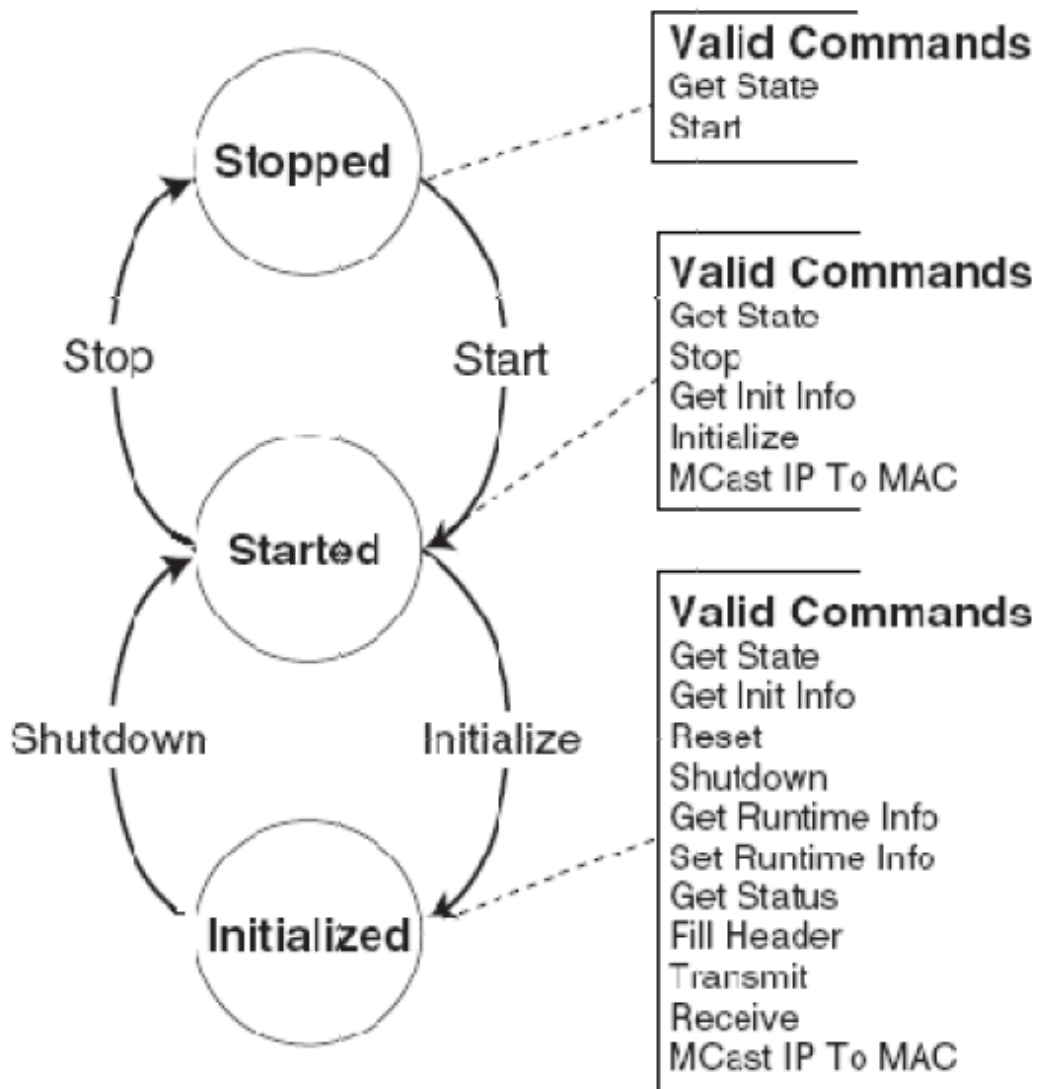


Fig. E.6: UNDI States, Transitions & Valid Commands

Additional requirements for S/W UNDI implementations: Processor register contents must be unchanged by S/W UNDI command execution (The application/driver does not have to save processor registers when calling S/W UNDI). Processor arithmetic flags are undefined (application/driver must save processor arithmetic flags if needed). Application/driver must remove CDB address from stack after control returns from S/W UNDI.

**Note**

Additional requirements for 32-bit network devices: All addresses given to the S/W UNDI must be 32-bit addresses. Any address that exceeds 32 bits (4 GiB) will result in a return of one of the following status codes: `PXE_STATCODE_INVALID_PARAMETER`, `PXE_STATCODE_INVALID_CDB` or `PXE_STATCODE_INVALID_CPB`.

When executing linked commands, command execution will stop at the end of the CDB list (when the `PXE_CONTROL_LINK` bit is not set) or when a command returns an error status code.

**Note**

Buffers requested via the `MemoryRequired` field in `s_pxe_db_get_init_info` (See [DB](#)) will be allocated via `PCI_IO.AllocateBuffer()`. However, the buffers passed to various UNDI commands are not guaranteed to be allocated via `AllocateBuffer()`.

**Note**

Calls to `Map_Mem()` of type `TO_AND_FROM_DEVICE` must only be used for common DMA buffers. Such buffers must be requested via the `MemoryRequired` field in `s_pxe_db_get_init_info` and provided through the `Initialize` command.

## E.4.1 Command Linking and Queuing

When linking commands, the CDBs must be stored consecutively in system memory without any gaps in between. Do not set the `Link` bit in the last CDB in the list. As shown in the Figure, below, the `Link` bit must be set in all other CDBs in the list.

When the H/W UNDI is executing commands, the `State` bits in the `Status` field in the `!PXE` structure will be set to `Busy` (3).

When H/W or S/W UNDI is executing commands and a new command is issued, a `StatCode` of `PXE_STATCODE_BUSY` and a `StatFlag` of `PXE_STATFLAG_COMMAND_FAILURE` is set in the CDB. For linked commands, only the first CDB will be set to `Busy`, all other CDBs will be unchanged. When a linked command fails, execution on the list stops. Commands after the failing command will not be run.

As shown in the Figure, below, when queuing commands, only the first CDB needs to have the `Queue Control` flag set. If queuing is supported and the UNDI is busy and there is room in the command queue, the command (or list of commands) will be queued.

When a command is queued a `StatFlag` of `PXE_STATFLAG_COMMAND_QUEUED` is set (if linked commands are queued only the `StatFlag` of the first CDB gets set). This signals that the command was added to the queue. Commands in the queue will be run on a first-in, first-out, basis. When a command fails, the next command in the queue is run. When a linked command in the queue fails, execution on the list stops. The next command, or list of commands, that was added to the command queue will be run.

| Linked CDBs |                      |
|-------------|----------------------|
| 0x00        | <b>CDB</b>           |
| 0x1F        | Set Link bit.        |
| 0x20        | <b>CDB</b>           |
| 0x3F        | Set Link bit.        |
| 0x40        | <b>CDB</b>           |
| 0x5F        | Do not set Link bit. |

OM13188

Fig. E.7: Linked CDBs

| Queued CDBs |                                 |
|-------------|---------------------------------|
| 0x00        | <b>CDB</b>                      |
| 0x1F        | Set Queue bit.<br>Set Link bit. |
| 0x20        | <b>CDB</b>                      |
| 0x3F        | Set Queue bit.<br>Set Link bit. |
| 0x40        | <b>CDB</b>                      |
| 0x5F        | Set Queue bit.<br>Set Link bit. |

Fig. E.8: Queued CDBs



## E.4.2 Get State

This command is used to determine the operational state of the UNDI. An UNDI has three possible operational states:

- **Stopped.** A stopped UNDI is free for the taking. When all interface numbers (IFnum) for a particular S/W UNDI are stopped, that S/W UNDI image can be relocated or removed. A stopped UNDI will accept Get State and Start commands.
- **Started.** A started UNDI is in use. A started UNDI will accept Get State, Stop, Get Init Info, and Initialize commands.
- **Initialized.** An initialized UNDI is in used. An initialized UNDI will accept all commands except: Start, Stop, and Initialize.

Drivers and applications must not start using UNDI that have been placed into the Started or Initialized states by another driver or application.

3.0 and 3.1 S/W UNDI: No callbacks are performed by this UNDI command.

### E.4.2.1 Issuing the Command

To issue a Get State command, create a CDB and fill it in as shown in the table below:

| CDB Field | How to initialize the CDB structure for a Get State command                   |
|-----------|-------------------------------------------------------------------------------|
| OpCode    | PXE_OPCODE_GET_STATE                                                          |
| OpFlags   | PXE_OPFLAGS_NOT_USED                                                          |
| CPBsize   | PXE_CPBSIZE_NOT_USED                                                          |
| DBsize    | PXE_DBSIZE_NOT_USED                                                           |
| CPBaddr   | PXE_CPBADDR_NOT_USED                                                          |
| DBaddr    | PXE_DBADDR_NOT_USED                                                           |
| StatCode  | PXE_STATCODE_INITIALIZE                                                       |
| StatFlags | PXE_STATFLAGS_INITIALIZE                                                      |
| IFnum     | A valid interface number from zero to ( !PXE.IFcnt   (!PXE.IFcntExt << 8 ) ). |
| Control   | Set as needed                                                                 |

### E.4.2.2 Waiting for the Command to Execute

Monitor the upper two bits (14 & 15) in the *CDB.StatFlags* field. Until these bits change to report *PXE\_STATFLAGS\_COMMAND\_COMPLETE* or *PXE\_STATFLAGS\_COMMAND\_FAILED*, the command has not been executed by the UNDI.

| StatFlags        | Reason                                                               |
|------------------|----------------------------------------------------------------------|
| COMMAND_COMPLETE | Command completed successfully. StatFlags contain operational state. |
| COMMAND_FAILED   | Command failed. StatCode field contains error code.                  |
| COMMAND_QUEUED   | Command has been queued. All other fields are unchanged.             |
| INITIALIZE       | Command has not been executed or queued.                             |

### E.4.2.3 Checking Command Execution Results

After command execution completes, either successfully or not, the *CDB.StatCode* field contains the result of the command execution.

| StatCode    | Reason                                                               |
|-------------|----------------------------------------------------------------------|
| SUCCESS     | Command completed successfully. StatFlags contain operational state. |
| INVALID_CDB | One of the CDB fields was not set correctly.                         |
| BUSY        | UNDI is already processing commands. Try again later.                |
| QUEUE_FULL  | Command queue is full. Try again later.                              |

If the command completes successfully, use *PXE\_STATFLAGS\_GET\_STATE\_MASK* to check the state of the UNDI.

| StatFlags   | Reason                                    |
|-------------|-------------------------------------------|
| STOPPED     | The UNDI is stopped.                      |
| STARTED     | The UNDI is started, but not initialized. |
| INITIALIZED | The UNDI is initialized.                  |

### E.4.3 Start

This command is used to change the UNDI operational state from stopped to started. No other operational checks are made by this command. Protocol driver makes this call for each network interface supported by the UNDI with a set of call back routines and a unique identifier to identify the particular interface. UNDI does not interpret the unique identifier in any way except that it is a 64-bit value and it will pass it back to the protocol driver as a parameter to all the call back routines for any particular interface. If this is a S/W UNDI, the callback functions Delay(), Virt2Phys(), Map\_Mem(), UnMap\_Mem(), and Sync\_Mem() functions will not be called by this command.

#### E.4.3.1 Issuing the Command

To issue a Start command for H/W UNDI, create a CDB and fill it in as shows in the table below:

| CDB Field | How to initialize the CDB structure for a H/W UNDI Start command              |
|-----------|-------------------------------------------------------------------------------|
| OpCode    | PXE_OPCODE_START                                                              |
| OpFlags   | PXE_OPFLAGS_NOT_USED                                                          |
| CPBsize   | PXE_CPBSIZE_NOT_USED                                                          |
| DBsize    | PXE_DBSIZE_NOT_USED                                                           |
| CPBaddr   | PXE_CPBADDR_NOT_USED                                                          |
| DBaddr    | PXE_DBADDR_NOT_USED                                                           |
| StatCode  | PXE_STATCODE_INITIALIZE                                                       |
| StatFlags | PXE_STATFLAGS_INITIALIZE                                                      |
| IFnum     | A valid interface number from zero to ( !PXE.IFcnt   (!PXE.IFcntExt << 8 ) ). |
| Control   | Set as needed                                                                 |

To issue a Start command for S/W UNDI, create a CDB and fill it in as shows in the table below:

| CDB Field | How to initialize the CDB structure for a S/W UNDI Start command |
|-----------|------------------------------------------------------------------|
| OpCode    | PXE_OPCODE_START                                                 |
| OpFlags   | PXE_OPFLAGS_NOT_USED                                             |

continues on next page

Table E.9 – continued from previous page

|           |                                                                               |
|-----------|-------------------------------------------------------------------------------|
| CPBsize   | sizeof(PXE_CPB_START)                                                         |
| DBsize    | PXE_DBSIZE_NOT_USED                                                           |
| CPBaddr   | Address of a PXE_CPB_START structure.                                         |
| DBaddr    | PXE_DBADDR_NOT_USED                                                           |
| StatCode  | PXE_STATCODE_INITIALIZE                                                       |
| StatFlags | PXE_STATFLAGS_INITIALIZE                                                      |
| IFnum     | A valid interface number from zero to ( !PXE.IFcnt   (!PXE.IFcntExt << 8 ) ). |
| Control   | Set as needed                                                                 |

### E.4.3.2 Preparing the CPB

For the 3.1 S/W UNDI Start command, the CPB structure shown below must be filled in and the CDB must be set to *sizeof(struct s\_pxe\_cpb\_start\_31)*.

```
#pragma pack(1)
typedef struct s_pxe_cpb_start_31 {
    UINT64 Delay;
    //
    // Address of the Delay() callback service.
    // This field cannot be set to zero.
    //
    // VOID
    // Delay(
    // IN UINT64 UniqueId,
    // IN UINT64 Microseconds);
    //
    // UNDI will never request a delay smaller than 10 microseconds
    // and will always request delays in increments of 10
    // microseconds. The Delay() callback routine must delay
    // between n and n + 10 microseconds before returning control
    // to the UNDI.
    //
    UINT64 Block;
    //
    // Address of the Block() callback service.
    // This field cannot be set to zero.
    //
    // VOID
    // Block(
    // IN UINT64 UniqueId,
    // IN UINT32 Enable);
    //
    // UNDI may need to block multithreaded/multiprocessor access
    // to critical code sections when programming or accessing the
    // network device. When UNDI needs a block, it will call the
    // Block()callback service with Enable set to a non-zero value.
    // When UNDI no longer needs the block, it will call Block()
    // with Enable set to zero.
    //
}
```

(continues on next page)

(continued from previous page)

```

UINT64 Virt2Phys;
//
// Convert a virtual address to a physical address.
// This field can be set to zero if virtual and physical
// addresses are identical.
//
// VOID
// Virt2Phys(
// IN UINT64 UniqueId,
// IN UINT64 Virtual,
// OUT UINT64 PhysicalPtr);
//
// UNDI will pass in a virtual address and a pointer to storage
// for a physical address. The Virt2Phys() service converts
// the virtual address to a physical address and stores the
// resulting physical address in the supplied buffer. If no
// conversion is needed, the virtual address must be copied
// into the supplied physical address buffer.
//

UINT64 Mem_IO;
//
// Read/Write network device memory and/or I/O register space.
// This field cannot be set to zero.
//
// VOID
// Mem_IO(
// IN UINT64 UniqueId,
// IN UINT8 AccessType,
// IN UINT8 Length,
// IN UINT64 Port,
// IN OUT UINT64 BufferPtr);
//
// UNDI uses the Mem_IO() service to access the network device
// memory and/or I/O registers. The AccessType is one of the
// PXE_IO_xxx or PXE_MEM_xxx constants defined at the end of
// this section. The Length is 1, 2, 4 or 8. The Port number
// is relative to the base memory or I/O address space for this
// device. BufferPtr points to the data to be written to the
// Port or will contain the data that is read from the Port.
//

UINT64 Map_Mem;
//
// Map virtual memory address for DMA.
// This field can be set to zero if there is no mapping
// service.
//
// VOID
// Map_Mem(
// IN UINT64 UniqueId,
// IN UINT64 Virtual,

```

(continues on next page)

(continued from previous page)

```

// IN UINT32 Size,
// IN UINT32 Direction,
// OUT UINT64 PhysicalPtr);
//
// When UNDI needs to perform a DMA transfer it will request a
// virtual-to-physical mapping using the Map_Mem() service. The
// Virtual parameter contains the virtual address to be mapped.
// The minimum Size of the virtual memory buffer to be mapped.
// Direction is one of the TO_DEVICE, FROM_DEVICE or
// TO_AND_FROM_DEVICE constants defined at the end of this
// section. PhysicalPtr contains the mapped physical address or
// a copy of the Virtual address if no mapping is required.
//
UINT64 UnMap_Mem;
//
// Un-map previously mapped virtual memory address.
// This field can be set to zero only if the Map_Mem() service
// is also set to zero.
//
// VOID
// UnMap_Mem(
// IN UINT64 UniqueId,
// IN UINT64 Virtual,
// IN UINT32 Size,
// IN UINT32 Direction,
// IN UINT64 PhysicalPtr);
//
// When UNDI is done with the mapped memory, it will use the
// UnMap_Mem() service to release the mapped memory.
//
UINT64 Sync_Mem;
//
// Synchronise mapped memory.
// This field can be set to zero only if the Map_Mem() service
// is also set to zero.
//
// VOID
// Sync_Mem(
// IN UINT64 UniqueId,
// IN UINT64 Virtual,
// IN UINT32 Size,
// IN UINT32 Direction,
// IN UINT64 PhysicalPtr);
//
// When the virtual and physical buffers need to be
// synchronized, UNDI will call the Sync_Mem() service.
//
UINT64 UniqueId;
//

```

(continues on next page)

(continued from previous page)

```

// UNDI will pass this value to each of the callback services.
// A unique ID number should be generated for each instance of
// the UNDI driver that will be using these callback services.
//
}   PXE_CPB_START_31;
#pragma pack()

```

For the 3.0 S/W UNDI Start command, the CPB structure shown below must be filled in and the CDB must be set to `sizeof(struct s_pxe_cpb_start_30)`.

```

#pragma pack(1)
typedef struct s_pxe_cpb_start_30 {
    UINT64 Delay;
    //
    // Address of the Delay() callback service.
    // This field cannot be set to zero.
    //
    // VOID
    // Delay(
    // IN UINT64 Microseconds);
    //
    // UNDI will never request a delay smaller than 10 microseconds
    // and will always request delays in increments of 10.
    // microseconds The Delay() callback routine must delay between
    // n and n + 10 microseconds before returning control to the
    // UNDI.
    //
    //
    UINT64 Block;
    //
    // Address of the Block() callback service.
    // This field cannot be set to zero.
    //
    // VOID
    // Block(
    // IN UINT32 Enable);
    //
    // UNDI may need to block multithreaded/multiprocessor access
    // to critical code sections when programming or accessing the
    // network device. When UNDI needs a block, it will call the
    // Block()callback service with Enable set to a non-zero value.
    // When UNDI no longer needs the block, it will call Block()
    // with Enable set to zero.
    //
    //
    UINT64 Virt2Phys;
    //
    // Convert a virtual address to a physical address.
    // This field can be set to zero if virtual and physical
    // addresses are identical.
    //
    //
    // VOID

```

(continues on next page)

(continued from previous page)

```

// Virt2Phys(
// IN UINT64 Virtual,
// OUT UINT64 PhysicalPtr);
//
// UNDI will pass in a virtual address and a pointer to storage
// for a physical address. The Virt2Phys() service converts
// the virtual address to a physical address and stores the
// resulting physical address in the supplied buffer. If no
// conversion is needed, the virtual address must be copied
// into the supplied physical address buffer.
//

UINT64 Mem_IO;
//
// Read/Write network device memory and/or I/O register space.
// This field cannot be set to zero.
//
// VOID
// Mem_IO(
// IN UINT8 AccessType,
// IN UINT8 Length,
// IN UINT64 Port,
// IN OUT UINT64 BufferPtr);
//
// UNDI uses the Mem_IO() service to access the network device
// memory and/or I/O registers. The AccessType is one of the
// PXE_IO_xxx or PXE_MEM_xxx constants defined at the end of
// this section. The Length is 1, 2, 4 or 8. The Port number
// is relative to the base memory or I/O address space for this
// device. BufferPtr points to the data to be written to the
// Port or will contain the data that is read from the Port.
//
} PXE_CPB_START_30;
#pragma pack()

#define TO_AND_FROM_DEVICE 0
// Provides both read and write access to system memory by both
// the processor and a bus master. The buffer is coherent from
// both the processor's and the bus master's point of view.

#define FROM_DEVICE 1
// Provides a write operation to system memory by a bus master.

#define TO_DEVICE 2
// Provides a read operation from system memory by a bus master.

```

### E.4.3.3 Waiting for the Command to Execute

Monitor the upper two bits (14 & 15) in the *CDB.StatFlags* field. Until these bits change to report *PXE\_STATFLAGS\_COMMAND\_COMPLETE* or *PXE\_STATFLAGS\_COMMAND\_FAILED*, the command has not been executed by the UNDI.

| StatFlags        | Reason                                               |
|------------------|------------------------------------------------------|
| COMMAND_COMPLETE | Command completed successfully. UNDI is now started. |
| COMMAND_FAILED   | Command failed. StatCode field contains error code.  |
| COMMAND_QUEUED   | Command has been queued.                             |
| INITIALIZE       | Command has been not executed or queued.             |

### E.4.3.4 Checking Command Execution Results

After command execution completes, either successfully or not, the *CDB.StatCode* field contains the result of the command execution.

| StatCode        | Reason                                                |
|-----------------|-------------------------------------------------------|
| SUCCESS         | Command completed successfully. UNDI is now started.  |
| INVALID_CDB     | One of the CDB fields was not set correctly.          |
| BUSY            | UNDI is already processing commands. Try again later. |
| QUEUE_FULL      | Command queue is full. Try again later.               |
| ALREADY_STARTED | The UNDI is already started.                          |

## E.4.4 Stop

This command is used to change the UNDI operational state from started to stopped.

### E.4.4.1 Issuing the Command

To issue a Stop command, create a CDB and fill it in as shows in the table below:

| CDB Field | How to initialize the CDB structure for a Stop command                                      |
|-----------|---------------------------------------------------------------------------------------------|
| OpCode    | PXE_OPCODE_STOP                                                                             |
| OpFlags   | PXE_OPFLAGS_NOT_USED                                                                        |
| CPBsize   | PXE_CPBSIZE_NOT_USED                                                                        |
| DBsize    | PXE_DBSIZE_NOT_USED                                                                         |
| CPBaddr   | PXE_CPBADDR_NOT_USED                                                                        |
| DBaddr    | PXE_DBADDR_NOT_USE                                                                          |
| StatCode  | PXE_STATCODE_INITIALIZE                                                                     |
| StatFlags | PXE_STATFLAGS_INITIALIZ                                                                     |
| IFnum     | A valid interface number from zero to ( <i>!PXE.IFcnt</i>   ( <i>!PXE.IFcntExt</i> << 8) ). |
| Control   | Set as needed                                                                               |



#### E.4.4.2 Waiting for the Command to Execute

Monitor the upper two bits (14 & 15) in the *CDB.StatFlags* field. Until these bits change to report *PXE\_STATFLAGS\_COMMAND\_COMPLETE* or *PXE\_STATFLAGS\_COMMAND\_FAILED*, the command has not been executed by the UNDI.

| StatFlags        | Reason                                               |
|------------------|------------------------------------------------------|
| COMMAND_COMPLETE | Command completed successfully. UNDI is now stopped. |
| COMMAND_FAILED   | Command failed. StatCode field contains error code.  |
| COMMAND_QUEUED   | Command has been queued.                             |
| INITIALIZE       | Command has not been executed or queued.             |

#### E.4.4.3 Checking Command Execution Results

After command execution completes, either successfully or not, the *CDB.StatCode* field contains the result of the command execution.

| StatCode     | Reason                                                                 |
|--------------|------------------------------------------------------------------------|
| SUCCESS      | Command completed successfully. UNDI is now stopped.                   |
| INVALID_CDB  | One of the CDB fields was not set correctly.                           |
| BUSY         | UNDI is already processing commands. Try again later.                  |
| QUEUE_FULL   | Command queue is full. Try again later.                                |
| NOT_STARTED  | The UNDI is not started.                                               |
| NOT_SHUTDOWN | The UNDI is initialized and must be shutdown before it can be stopped. |

### E.4.5 Get Init Info

This command is used to retrieve initialization information that is needed by drivers and applications to initialize UNDI.

#### E.4.5.1 Issuing the Command

To issue a Get Init Info command, create a CDB and fill it in as shows in the table below:

| CDB Field | How to initialize the CDB structure for a Get Init Info command                             |
|-----------|---------------------------------------------------------------------------------------------|
| OpCode    | <i>PXE_OPCODE_GET_INIT_INFO</i>                                                             |
| OpFlags   | <i>PXE_OPFLAGS_NOT_USED</i>                                                                 |
| CPBsize   | <i>PXE_CPBSIZE_NOT_USED</i>                                                                 |
| DBsize    | <i>sizeof(PXE_DB_INIT_INFO)</i>                                                             |
| CPBaddr   | <i>PXE_CPBADDR_NOT_USED</i>                                                                 |
| DBaddr    | Address of a <i>PXE_DB_INIT_INFO</i> structure.                                             |
| StatCode  | <i>PXE_STATCODE_INITIALIZE</i>                                                              |
| StatFlags | <i>PXE_STATFLAGS_INITIALIZE</i>                                                             |
| IFnum     | A valid interface number from zero to ( <i>!PXE.IFcnt</i>   ( <i>!PXE.IFcntExt</i> << 8) ). |
| Control   | Set as needed.                                                                              |

### E.4.5.2 Waiting for the Command to Execute

Monitor the upper two bits (14 & 15) in the *CDB.StatFlags* field. Until these bits change to report *PXE\_STATFLAGS\_COMMAND\_COMPLETE* or *PXE\_STATFLAGS\_COMMAND\_FAILED*, the command has not been executed by the UNDI.

| StatFlags        | Reason                                              |
|------------------|-----------------------------------------------------|
| COMMAND_COMPLETE | Command completed successfully. DB can be used.     |
| COMMAND_FAILED   | Command failed. StatCode field contains error code. |
| COMMAND_QUEUED   | Command has been queued.                            |
| INITIALIZE       | Command has been not executed or queued.            |

### E.4.5.3 Checking Command Execution Results

After command execution completes, either successfully or not, the *CDB.StatCode* field contains the result of the command execution.

| StatCode    | Reason                                                |
|-------------|-------------------------------------------------------|
| SUCCESS     | Command completed successfully. DB can be used.       |
| INVALID_CDB | One of the CDB fields was not set correctly.          |
| BUSY        | UNDI is already processing commands. Try again later. |
| QUEUE_FULL  | Command queue is full. Try again later.               |
| NOT_STARTED | The UNDI is not started.                              |

### E.4.5.4 StatFlags

To determine if cable detection is supported by this UNDI/NIC, use these macros with the value returned in the *CDB.StatFlags* field:

```
PXE_STATFLAGS_CABLE_DETECT_MASK
PXE_STATFLAGS_CABLE_DETECT_NOT_SUPPORTED
PXE_STATFLAGS_CABLE_DETECT_SUPPORTED
PXE_STATFLAGS_GET_STATUS_NO_MEDIA_MASK
PXE_STATFLAGS_GET_STATUS_NO_MEDIA_NOT_SUPPORTED
PXE_STATFLAGS_GET_STATUS_NO_MEDIA_SUPPORTED
```

### E.4.5.5 DB

```
#pragma pack(1)
typedef struct s_pxe_db_get_init_info {

    // Minimum length of locked memory buffer that must be given to
    // the Initialize command. Giving UNDI more memory will
    // generally give better performance.

    // If MemoryRequired is zero, the UNDI does not need and will
    // not use system memory to receive and transmit packets.
```

(continues on next page)

(continued from previous page)

```

PXE_UINT32 MemoryRequired;

// Maximum frame data length for Tx/Rx excluding the media
// header.
//
PXE_UINT32 FrameDataLen;

// Supported link speeds are in units of mega bits. Common
// ethernet values are 10, 100 and 1000. Unused LinkSpeeds[]
// entries are zero filled.

PXE_UINT32 LinkSpeeds[4];

// Number of nonvolatile storage items.

PXE_UINT32 NvCount;

// Width of nonvolatile storage item in bytes. 0, 1, 2 or 4

PXE_UINT16 NvWidth;

// Media header length. This is the typical media header
// length for this UNDI. This information is needed when
// allocating receive and transmit buffers.

PXE_UINT16 MediaHeaderLen;

// Number of bytes in the NIC hardware (MAC) address.

PXE_UINT16 HWAddrLen;

// Maximum number of multicast MAC addresses in the multicast
// MAC address filter list.

PXE_UINT16 MCastFilterCnt;

// Default number and size of transmit and receive buffers that
// will be allocated by the UNDI. If MemoryRequired is
// nonzero, this allocation will come out of the memory buffer
// given to the Initialize command. If MemoryRequired is zero,
// this allocation will come out of memory on the NIC.

PXE_UINT16 TxBufCnt;
PXE_UINT16 TxBufSize;
PXE_UINT16 RxBufCnt;
PXE_UINT16 RxBufSize;

// Hardware interface types defined in the Assigned Numbers RFC
// and used in DHCP and ARP packets.
// See the PXE_IFTYPE typedef and PXE_IFTYPE_xxx macros.

PXE_UINT8 IFtype;

```

(continues on next page)

(continued from previous page)

```

// Supported duplex options. This can be one or a combination
// of more than one constants defined as PXE_DUPLEX_xxxxx
// below. This value indicates the ability of UNDI to
// change/control the duplex modes of the NIC.

PXE_UINT8 SupportedDuplexModes;

// Supported loopback options. This field can be one or a
// combination of more than one constants defined as
// PXE_LOOPBACK_xxxxx #defines below. This value indicates
// the ability of UNDI to change/control the loopback modes
// of the NIC

PXE_UINT8 SupportedLoopBackModes;
} PXE_DB_GET_INIT_INFO;
#pragma pack()

#define PXE_MAX_TXRX_UNIT_ETHER 1500
#define PXE_HWADDR_LEN_ETHER 0x0006

#define PXE_DUPLEX_DEFAULT 0
#define PXE_DUPLEX_ENABLE_FULL_SUPPORTED 1
#define PXE_DUPLEX_FORCE_FULL_SUPPORTED 2

#define PXE_LOOPBACK_INTERNAL_SUPPORTED 1
#define PXE_LOOPBACK_EXTERNAL_SUPPORTED 2

```

## E.4.6 Get Config Info

This command is used to retrieve configuration information about the NIC being controlled by the UNDI.

### E.4.6.1 Issuing the Command

To issue a Get Config Info command, create a CDB and fill it in as shown in the table below:

| CDB Field | How to initialize the CDB structure for a Get Config Info command          |
|-----------|----------------------------------------------------------------------------|
| OpCode    | PXE_OPCODE_GET_CONFIG_INFO                                                 |
| OpFlags   | PXE_OPFLAGS_NOT_USED                                                       |
| CPBsize   | PXE_CPBSIZE_NOT_USED                                                       |
| DBsize    | sizeof(PXE_DB_CONFIG_INFO)                                                 |
| CPBaddr   | PXE_CPBADDR_NOT_USED                                                       |
| DBaddr    | Address of a PXE_DB_CONFIG_INFO structure                                  |
| StatCode  | PXE_STATCODE_INITIALIZE                                                    |
| StatFlags | PXE_STATFLAGS_INITIALIZE                                                   |
| IFnum     | A valid interface number from zero to (!PXE.IFcnt   (!PXE.IFcntExt << 8)). |
| Control   | Set as needed                                                              |

### E.4.6.2 Waiting for the Command to Execute

Monitor the upper two bits (14 & 15) in the *CDB.StatFlags* field. Until these bits change to report *PXE\_STATFLAGS\_COMMAND\_COMPLETE* or *PXE\_STATFLAGS\_COMMAND\_FAILED*, the command has not been executed by the UNDI.

| StatFlags        | Reason                                               |
|------------------|------------------------------------------------------|
| COMMAND_COMPLETE | Command completed successfully. DB has been written. |
| COMMAND_FAILED   | Command failed. StatCode field contains error code.  |
| COMMAND_QUEUED   | Command has been queued.                             |
| INITIALIZE       | Command has been not executed or queued.             |

### E.4.6.3 Checking Command Execution Results

After command execution completes, either successfully or not, the *CDB.StatCode* field contains the result of the command execution.

| StatCode    | Reason                                                |
|-------------|-------------------------------------------------------|
| SUCCESS     | Command completed successfully. DB has been written.  |
| INVALID_CDB | One of the CDB fields was not set correctly.          |
| BUSY        | UNDI is already processing commands. Try again later. |
| QUEUE_FULL  | Command queue is full. Try again later.               |
| NOT_STARTED | The UNDI is not started.                              |

### E.4.6.4 DB

```
#pragma pack(1)
typedef struct s_pxe_pci_config_info {

    // This is the flag field for the PXE_DB_GET_CONFIG_INFO union.
    // For PCI bus devices, this field is set to PXE_BUSTYPE_PCI.

    PXE_UINT32 BusType;

    // This identifies the PCI network device that this UNDI
    // interface is bound to.

    PXE_UINT16 Bus;
    PXE_UINT8 Device;
    PXE_UINT8 Function;

    // This is a copy of the PCI configuration space for this
    // network device.

    union {
        PXE_UINT8 Byte[256];
        PXE_UINT16 Word[128];
    };
};
```

(continues on next page)

(continued from previous page)

```

    PXE_UINT32 Dword[64];
    } Config;
} PXE_PCI_CONFIG_INFO;
#pragma pack()
#pragma pack(1)
typedef struct s_pxe_pcc_config_info {

    // This is the flag field for the PXE_DB_GET_CONFIG_INFO union.
    // For PCC bus devices, this field is set to PXE_BUSTYPE_PCC.

    PXE_UINT32 BusType;

    // This identifies the PCC network device that this UNDI
    // interface is bound to.

    PXE_UINT16 Bus;
    PXE_UINT8 Device;
    PXE_UINT8 Function;

    // This is a copy of the PCC configuration space for this
    // network device.

    union {
        PXE_UINT8 Byte[256];
        PXE_UINT16 Word[128];
        PXE_UINT32 Dword[64];
    } Config;
} PXE_PCC_CONFIG_INFO;
#pragma pack()

#pragma pack(1)
typedef union u_pxe_db_get_config_info {
    PXE_PCI_CONFIG_INFO pci;
    PXE_PCC_CONFIG_INFO pcc;
} PXE_DB_GET_CONFIG_INFO;
#pragma pack()

```

## E.4.7 Initialize

This command resets the network adapter and initializes UNDI using the parameters supplied in the CPB. The Initialize command must be issued before the network adapter can be setup to transmit and receive packets. This command will not enable the receive unit or external interrupts.

Once the memory requirements of the UNDI are obtained by using the Get Init Info command, a block of kernel (nonswappable) memory may need to be allocated by the protocol driver. The address of this kernel memory must be passed to UNDI using the Initialize command CPB. This memory is used for transmit and receive buffers and internal processing.

Initializing the network device will take up to four seconds for most network devices and in some extreme cases (usually poor cables) up to twenty seconds. Control will not be returned to the caller and the *COMMAND\_COMPLETE* status flag will not be set until the NIC is ready to transmit.

### E.4.7.1 Issuing the Command

To issue an Initialize command, create a CDB and fill it in as shows in the table below:

| CDB Field | How to initialize the CDB structure for an Initialize command                               |
|-----------|---------------------------------------------------------------------------------------------|
| OpCode    | PXE_OPCODE_INITIALIZE                                                                       |
| CPBsize   | sizeof(PXE_CPB_INITIALIZE)                                                                  |
| DBsize    | sizeof(PXE_DB_INITIALIZE)                                                                   |
| CPBaddr   | Address of a PXE_CPB_INITIALIZE structure.                                                  |
| Dbaddr    | Address of a PXE_DB_INITIALIZE structure.                                                   |
| StatCode  | PXE_STATCODE_INITIALIZE                                                                     |
| StatFlags | PXE_STATFLAGS_INITIALIZE                                                                    |
| Ifnum     | A valid interface number from zero to ( <i>!PXE.IFcnt</i>   ( <i>!PXE.IFcntExt</i> << 8) ). |
| Control   | Set as needed.                                                                              |

### E.4.7.2 OpFlags

Cable detection can be enabled or disabled by setting one of the following OpFlags:

PXE\_OPFLAGS\_INITIALIZE\_CABLE\_DETECT  
PXE\_OPFLAGS\_INITIALIZE\_DO\_NOT\_DETECT\_CABLE

### E.4.7.3 Preparing the CPB

If the *MemoryRequired* field returned in the *PXE\_DB\_GET\_INIT\_INFO* structure is zero, the Initialize command does not need to be given a memory buffer or even a CPB structure. If the *MemoryRequired* field is nonzero, the Initialize command does need a memory buffer.

```
#pragma pack(1)
typedef struct s_pxe_cpb_initialize {

    // Address of first (lowest) byte of the memory buffer.
    // This buffer must be in contiguous physical memory and cannot
    // be swapped out. The UNDI will be using this for transmit
    // and receive buffering. This address must be a processor-
    // based address for S/W UNDI and a device-based address for
    // H/W UNDI.

    PXE_UINT64 MemoryAddr;

    // MemoryLength must be greater than or equal to MemoryRequired
    // returned by the Get Init Info command.

    PXE_UINT32 MemoryLength;

    // Desired link speed in Mbit/sec. Common ethernet values are
    // 10, 100 and 1000. Setting a value of zero will auto-detect
    // and/or use the default link speed (operation depends on
    // UNDI/NIC functionality).

    PXE_UINT32 LinkSpeed;
```

(continues on next page)

(continued from previous page)

```

// Suggested number and size of receive and transmit buffers to
// allocate. If MemoryAddr and MemoryLength are nonzero, this
// allocation comes out of the supplied memory buffer. If
// MemoryAddr and MemoryLength are zero, this allocation comes
// out of memory on the NIC.

// If these fields are set to zero, the UNDI will allocate
// buffer counts and sizes as it sees fit.

PXE_UINT16 TxBufCnt;
PXE_UINT16 TxBufSize;
PXE_UINT16 RxBufCnt;
PXE_UINT16 RxBufSize;

// The following configuration parameters are optional and must
// be zero to use the default values.
// The possible values for these parameters are defined below.

PXE_UINT8 DuplexMode;

PXE_UINT8 LoopBackMode;

} PXE_CPB_INITIALIZE;
#pragma pack()

#define PXE_DUPLEX_AUTO_DETECT 0x00
#define PXE_FORCE_FULL_DUPLEX 0x01

#define PXE_FORCE_HALF_DUPLEX 0x02
#define PXE_LOOPBACK_NORMAL 0
#define PXE_LOOPBACK_INTERNAL 1
#define PXE_LOOPBACK_EXTERNAL 2

```

#### E.4.7.4 Waiting for the Command to Execute

Monitor the upper two bits (14 & 15) in the *CDB.StatFlags* field. Until these bits change to report *PXE\_STATFLAGS\_COMMAND\_COMPLETE* or *PXE\_STATFLAGS\_COMMAND\_FAILED*, the command has not been executed by the UNDI.

| StatFlags        | Reason                                                                                           |
|------------------|--------------------------------------------------------------------------------------------------|
| COMMAND_COMPLETE | Command completed successfully. UNDI and network device is now initialized. DB has been written. |
| COMMAND_FAILED   | Command failed. StatCode field contains error code.                                              |
| COMMAND_QUEUED   | Command has been queued.                                                                         |
| INITIALIZE       | Command has been not executed or queued.                                                         |



#### E.4.7.5 Checking Command Execution Results

After command execution completes, either successfully or not, the *CDB.StatCode* field contains the result of the command execution.

| StatCode            | Reason                                                                                                            |
|---------------------|-------------------------------------------------------------------------------------------------------------------|
| SUCCESS             | Command completed successfully. UNDI and network device is now initialized. DB has been written. Check StatFlags. |
| INVALID_CDB         | One of the CDB fields was not set correctly.                                                                      |
| INVALID_CPB         | One of the CPB fields was not set correctly.                                                                      |
| BUSY                | UNDI is already processing commands. Try again later.                                                             |
| QUEUE_FULL          | Command queue is full. Try again later.                                                                           |
| NOT_STARTED         | The UNDI is not started.                                                                                          |
| ALREADY_INITIALIZED | The UNDI is already initialized.                                                                                  |
| DEVICE_FAILURE      | The network device could not be initialized.                                                                      |
| NVDATA_FAILURE      | The nonvolatile storage could not be read.                                                                        |

#### E.4.7.6 StatFlags

Check the StatFlags to see if there is an active connection to this network device. If the no media StatFlag is set, the UNDI and network device are still initialized. *PXE\_STATFLAGS\_INITIALIZED\_NO\_MEDIA*

#### E.4.7.7 Before Using the DB

```
#pragma pack(1)
typedef struct s_pxe_db_initialize {

    // Actual amount of memory used from the supplied memory
    // buffer. This may be less than the amount of memory
    // supplied and may be zero if the UNDI and network device
    // do not use external memory buffers. Memory used by the
    // UNDI and network device is allocated from the lowest
    // memory buffer address.

    PXE_UINT32 MemoryUsed;

    // Actual number and size of receive and transmit buffers that
    // were allocated.

    PXE_UINT16 TxBufCnt;
    PXE_UINT16 TxBufSize;
    PXE_UINT16 RxBufCnt;
    PXE_UINT16 RxBufSize
} PXE_DB_INITIALIZE;
#pragma pack()
```

## E.4.8 Reset

This command resets the network adapter and reinitializes the UNDI with the same parameters provided in the Initialize command. The transmit and receive queues are emptied and any pending interrupts are cleared. Depending on the state of the OpFlags, the receive filters and external interrupt enables may also be reset.

Resetting the network device may take up to four seconds and in some extreme cases (usually poor cables) up to twenty seconds. Control will not be returned to the caller and the *COMMAND\_COMPLETE* status flag will not be set until the NIC is ready to transmit.

### E.4.8.1 Issuing the Command

To issue a Reset command, create a CDB and fill it in as shows in the table below:

| CDB Field | How to initialize the CDB structure for a Reset command                       |
|-----------|-------------------------------------------------------------------------------|
| OpCode    | PXE_OPCODE_RESET                                                              |
| OpFlags   | Set as needed.                                                                |
| CPBsize   | PXE_CPBSIZE_NOT_USED                                                          |
| DBsize    | PXE_DBSIZE_NOT_USED                                                           |
| CPBaddr   | PXE_CPBSIZE_NOT_USED                                                          |
| DBaddr    | PXE_DBSIZE_NOT_USED                                                           |
| StatCode  | PXE_STATCODE_INITIALIZE                                                       |
| StatFlags | PXE_STATFLAGS_INITIALIZE                                                      |
| IFnum     | A valid interface number from zero to ( !PXE.IFcnt   (!PXE.IFcntExt << 8 ) ). |
| Control   | Set as needed.                                                                |

### E.4.8.2 OpFlags

Normally the settings of the receive filters and external interrupt enables are unchanged by the Reset command. These two OpFlags will alter the operation of the Reset command.

PXE\_OPFLAGS\_RESET\_DISABLE\_INTERRUPTS  
PXE\_OPFLAGS\_RESET\_DISABLE\_FILTERS

### E.4.8.3 Waiting for the Command to Execute

Monitor the upper two bits (14 & 15) in the *CDB.StatFlags* field. Until these bits change to report *PXE\_STATFLAGS\_COMMAND\_COMPLETE* or *PXE\_STATFLAGS\_COMMAND\_FAILED*, the command has not been executed by the UNDI.

| StatFlags        | Reason                                                                                    |
|------------------|-------------------------------------------------------------------------------------------|
| COMMAND_COMPLETE | Command completed successfully. UNDI and network device have been reset. Check StatFlags. |
| COMMAND_FAILED   | Command failed. StatCode field contains error code.                                       |
| COMMAND_QUEUED   | Command has been queued.                                                                  |
| INITIALIZE       | Command has been not executed or queued.                                                  |

#### E.4.8.4 Checking Command Execution Results

After command execution completes, either successfully or not, the *CDB.StatCode* field contains the result of the command execution.

| StatCode        | Reason                                                                                       |
|-----------------|----------------------------------------------------------------------------------------------|
| SUCCESS         | Command completed successfully.<br>UNDI and network device have been reset. Check StatFlags. |
| INVALID_CDB     | One of the CDB fields was not set correctly.                                                 |
| BUSY            | UNDI is already processing commands. Try again later.                                        |
| QUEUE_FULL      | Command queue is full. Try again later.                                                      |
| NOT_STARTED     | The UNDI is not started.                                                                     |
| NOT_INITIALIZED | The UNDI is not initialized.                                                                 |
| DEVICE_FAILURE  | The network device could not be initialized.                                                 |
| NVDATA_FAILURE  | The nonvolatile storage is not valid.                                                        |

#### E.4.8.5 StatFlags

Check the StatFlags to see if there is an active connection to this network device. If the no media StatFlag is set, the UNDI and network device are still reset. *PXE\_STATFLAGS\_RESET\_NO\_MEDIA*

### E.4.9 Shutdown

The Shutdown command resets the network adapter and leaves it in a safe state for another driver to initialize. Any pending transmits or receives are lost. Receive filters and external interrupt enables are reset (disabled). The memory buffer assigned in the Initialize command can be released or reassigned.

Once UNDI has been shutdown, it can then be stopped or initialized again. The Shutdown command changes the UNDI operational state from initialized to started.

#### E.4.9.1 Issuing the Command

To issue a Shutdown command, create a CDB and fill it in as shown in the table below:

| CDB Field | How to initialize the CDB structure for a Shutdown command                    |
|-----------|-------------------------------------------------------------------------------|
| OpCode    | PXE_OPCODE_SHUTDOWN                                                           |
| OpFlags   | PXE_OPFLAGS_NOT_USED                                                          |
| CPBsize   | PXE_CPBSIZE_NOT_USED                                                          |
| DBsize    | PXE_DBSIZE_NOT_USED                                                           |
| CPBaddr   | PXE_CPBSIZE_NOT_USED                                                          |
| DBaddr    | PXE_DBSIZE_NOT_USED                                                           |
| StatCode  | PXE_STATCODE_INITIALIZE                                                       |
| StatFlags | PXE_STATFLAGS_INITIALIZE                                                      |
| IFnum     | A valid interface number from zero to ( !PXE.IFcnt   (!PXE.IFcntExt << 8 ) ). |
| Control   | Set as needed.                                                                |

### E.4.9.2 Waiting for the Command to Execute

Monitor the upper two bits (14 & 15) in the *CDB.StatFlags* field. Until these bits change to report *PXE\_STATFLAGS\_COMMAND\_COMPLETE* or *PXE\_STATFLAGS\_COMMAND\_FAILED*, the command has not been executed by the UNDI.

| StatFlags        | Reason                                                                |
|------------------|-----------------------------------------------------------------------|
| COMMAND_COMPLETE | Command completed successfully. UNDI and network device are shutdown. |
| COMMAND_FAILED   | Command failed. StatCode field contains error code.                   |
| COMMAND_QUEUED   | Command has been queued.                                              |
| INITIALIZE       | Command has been not executed or queued.                              |

### E.4.9.3 Checking Command Execution Results

After command execution completes, either successfully or not, the *CDB.StatCode* field contains the result of the command execution.

| StatCode        | Reason                                                                |
|-----------------|-----------------------------------------------------------------------|
| SUCCESS         | Command completed successfully. UNDI and network device are shutdown. |
| INVALID_CDB     | One of the CDB fields was not set correctly.                          |
| BUSY            | UNDI is already processing commands. Try again later.                 |
| QUEUE_FULL      | Command queue is full. Try again later.                               |
| NOT_STARTED     | The UNDI is not started.                                              |
| NOT_INITIALIZED | The UNDI is not initialized.                                          |

## E.4.10 Interrupt Enables

The Interrupt Enables command can be used to read and/or change the current external interrupt enable settings. Disabling an external interrupt enable prevents an external (hardware) interrupt from being signaled by the network device, internally the interrupt events can still be polled by using the Get Status command.

### E.4.10.1 Issuing the Command

To issue an Interrupt Enables command, create a CDB and fill it in as shows in the table below:

| CDB Field | How to initialize the CDB structure for an Interrupt Enables command                        |
|-----------|---------------------------------------------------------------------------------------------|
| OpCode    | PXE_OPCODE_INTERRUPT_ENABLES                                                                |
| OpFlags   | Set as needed.                                                                              |
| CPBsize   | PXE_CPBSIZE_NOT_USED                                                                        |
| DBsize    | PXE_DBSIZE_NOT_USED                                                                         |
| CPBaddr   | PXE_CPBADDR_NOT_USED                                                                        |
| DBaddr    | PXE_DBADDR_NOT_USED                                                                         |
| StatCode  | PXE_STATCODE_INITIALIZE                                                                     |
| StatFlags | PXE_STATFLAGS_INITIALIZE                                                                    |
| IFnum     | A valid interface number from zero to ( <i>!PXE.IFcnt</i>   ( <i>!PXE.IFcntExt</i> << 8) ). |
| Control   | Set as needed.                                                                              |

### E.4.10.2 OpFlags

To read the current external interrupt enables settings set *CDB.OpFlags* to:

PXE\_OPFLAGS\_INTERRUPT\_READ

To enable or disable external interrupts set one of these OpFlags:

PXE\_OPFLAGS\_INTERRUPT\_DISABLE  
PXE\_OPFLAGS\_INTERRUPT\_ENABLE

When enabling or disabling interrupt settings, the following additional OpFlag bits are used to specify which types of external interrupts are to be enabled or disabled:

PXE\_OPFLAGS\_INTERRUPT\_RECEIVE  
PXE\_OPFLAGS\_INTERRUPT\_TRANSMIT  
PXE\_OPFLAGS\_INTERRUPT\_COMMAND  
PXE\_OPFLAGS\_INTERRUPT\_SOFTWARE

Setting *PXE\_OPFLAGS\_INTERRUPT\_SOFTWARE* does not enable an external interrupt type, it generates an external interrupt.

### E.4.10.3 Waiting for the Command to Execute

Monitor the upper two bits (14 & 15) in the *CDB.StatFlags* field. Until these bits change to report *PXE\_STATFLAGS\_COMMAND\_COMPLETE* or *PXE\_STATFLAGS\_COMMAND\_FAILED*, the command has not been executed by the UNDI.

| StatFlags        | Reason                                              |
|------------------|-----------------------------------------------------|
| COMMAND_COMPLETE | Command completed successfully. Check StatFlags.    |
| COMMAND_FAILED   | Command failed. StatCode field contains error code. |
| COMMAND_QUEUED   | Command has been queued.                            |
| INITIALIZE       | Command has been not executed or queued.            |

### E.4.10.4 Checking Command Execution Results

After command execution completes, either successfully or not, the *CDB.StatCode* field contains the result of the command execution.

| StatCode        | Reason                                                |
|-----------------|-------------------------------------------------------|
| SUCCESS         | Command completed successfully. Check StatFlags.      |
| INVALID_CDB     | One of the CDB fields was not set correctly.          |
| BUSY            | UNDI is already processing commands. Try again later. |
| QUEUE_FULL      | Command queue is full. Try again later.               |
| NOT_STARTED     | The UNDI is not started.                              |
| NOT_INITIALIZED | The UNDI is not initialized.                          |

#### E.4.10.5 StatFlags

If the command was successful, the *CDB.StatFlags* field reports which external interrupt enable types are currently set. Possible *CDB.StatFlags* bit settings are:

- *PXE\_STATFLAGS\_INTERRUPT\_RECEIVE*
- *PXE\_STATFLAGS\_INTERRUPT\_TRANSMIT*
- *PXE\_STATFLAGS\_INTERRUPT\_COMMAND*

The bits set in *CDB.StatFlags* may be different than those that were requested in *CDB.OpFlags*. For example: If transmit and receive share an external interrupt line, setting either the transmit or receive interrupt will always enable both transmit and receive interrupts. In this case both transmit and receive interrupts will be reported in *CDB.StatFlags*. Always expect to get more than you ask for!

#### E.4.11 Receive Filters

This command is used to read and change receive filters and, if supported, read and change the multicast MAC address filter list. Control will not be returned to the caller and the *COMMAND\_COMPLETE* status flag will not be set until the NIC is ready to receive.

##### E.4.11.1 Issuing the Command

To issue a Receive Filters command, create a CDB and fill it in as shows in the table below:

| CDB Field | How to initialize the CDB structure for a Receive Filters command                 |
|-----------|-----------------------------------------------------------------------------------|
| OpCode    | <i>PXE_OPCODE_RECEIVE_FILTERS</i>                                                 |
| OpFlags   | Set as needed.                                                                    |
| CPBsize   | <code>sizeof(PXE_CPB_RECEIVE_FILTERS)</code>                                      |
| DBsize    | <code>sizeof(PXE_DB_RECEIVE_FILTERS)</code>                                       |
| CPBaddr   | Address of <i>PXE_CPB_RECEIVE_FILTERS</i> structure.                              |
| DBaddr    | Address of <i>PXE_DB_RECEIVE_FILTERS</i> structure.                               |
| StatCode  | <i>PXE_STATCODE_INITIALIZE</i>                                                    |
| StatFlags | <i>PXE_STATFLAGS_INITIALIZE</i>                                                   |
| IFnum     | A valid interface number from zero to $(!PXE.IFcnt \mid (!PXE.IFcntExt \ll 8))$ . |
| Control   | Set as needed.                                                                    |

##### E.4.11.2 OpFlags

To read the current receive filter settings set the *CDB.OpFlags* field to:

- *PXE\_OPFLAGS\_RECEIVE\_FILTER\_READ*

To change the current receive filter settings set one of these OpFlag bits:

- *PXE\_OPFLAGS\_RECEIVE\_FILTER\_ENABLE*
- *PXE\_OPFLAGS\_RECEIVE\_FILTER\_DISABLE*

When changing the receive filter settings, at least one of the OpFlag bits in this list must be selected:

- *PXE\_OPFLAGS\_RECEIVE\_FILTER\_UNICAST*
- *PXE\_OPFLAGS\_RECEIVE\_FILTER\_BROADCAST*
- *PXE\_OPFLAGS\_RECEIVE\_FILTER\_FILTERED\_MULTICAST*

- *PXE\_OPFLAGS\_RECEIVE\_FILTER\_PROMISCUOUS*
- *PXE\_OPFLAGS\_RECEIVE\_FILTER\_ALL\_MULTICAST*

To clear the contents of the multicast MAC address filter list, set this OpFlag:

- *PXE\_OPFLAGS\_RECEIVE\_FILTER\_RESET\_MCAST\_LIST*

### E.4.11.3 Preparing the CPB

The receive filter CPB is used to change the contents multicast MAC address filter list. To leave the multicast MAC address filter list unchanged, set the *CDB.CPBsize* field to *PXE\_CPBSIZE\_NOT\_USED* and *CDB.CPBaddr* to *PXE\_CPBADDR\_NOT\_USED*.

To change the multicast MAC address filter list, set *CDB.CPBsize* to the size, in bytes, of the multicast MAC address filter list and set *CDB.CPBaddr* to the address of the first entry in the multicast MAC address filter list.

```
typedef struct s_pxe_cpb_receive_filters {

    // List of multicast MAC addresses. This list, if present,
    // will replace the existing multicast MAC address filter list.

    PXE_MAC_ADDR MCastList[n];
} PXE_CPB_RECEIVE_FILTERS;
```

### E.4.11.4 Waiting for the Command to Execute

Monitor the upper two bits (14 & 15) in the *CDB.StatFlags* field. Until these bits change to report *PXE\_STATFLAGS\_COMMAND\_COMPLETE* or *PXE\_STATFLAGS\_COMMAND\_FAILED*, the command has not been executed by the UNDI.

| StatFlags        | Reason                                                          |
|------------------|-----------------------------------------------------------------|
| COMMAND_COMPLETE | Command completed successfully. Check StatFlags. DB is written. |
| COMMAND_FAILED   | Command failed. StatCode field contains error code.             |
| COMMAND_QUEUED   | Command has been queued.                                        |
| INITIALIZE       | Command has been not executed or queued.                        |

### E.4.11.5 Checking Command Execution Results

After command execution completes, either successfully or not, the *CDB.StatCode* field contains the result of the command execution.

| StatCode        | Reason                                                          |
|-----------------|-----------------------------------------------------------------|
| SUCCESS         | Command completed successfully. Check StatFlags. DB is written. |
| INVALID_CDB     | One of the CDB fields was not set correctly.                    |
| INVALID_CPB     | One of the CPB fields was not set correctly.                    |
| BUSY            | UNDI is already processing commands. Try again later.           |
| QUEUE_FULL      | Command queue is full. Try again later.                         |
| NOT_STARTED     | The UNDI is not started.                                        |
| NOT_INITIALIZED | The UNDI is not initialized.                                    |

### E.4.11.6 StatFlags

The receive filter settings in CDB.StatFlags are:

- *PXE\_STATFLAGS\_RECEIVE\_FILTER\_UNICAST*
- *PXE\_STATFLAGS\_RECEIVE\_FILTER\_BROADCAST*
- *PXE\_STATFLAGS\_RECEIVE\_FILTER\_FILTERED\_MULTICAST*
- *PXE\_STATFLAGS\_RECEIVE\_FILTER\_PROMISCUOUS*
- *PXE\_STATFLAGS\_RECEIVE\_FILTER\_ALL\_MULTICAST*

Unsupported receive filter settings in OpFlags are promoted to the next more liberal receive filter setting. For example: If broadcast or filtered multicast are requested and are not supported by the network device, but promiscuous is; the promiscuous status flag will be set.

### E.4.11.7 DB

The DB is used to read the current multicast MAC address filter list. The CDB.DBsize and CDB.DBaddr fields can be set to PXE\_DB\_SIZE\_NOT\_USED and PXE\_DBADDR\_NOT\_USED if the multicast MAC address filter list does not need to be read. When reading the multicast MAC address filter list extra entries in the DB will be filled with zero.

```
typedef struct s_pxe_db_receive_filters {

    // Filtered multicast MAC address list.

    PXE_MAC_ADDR MCastList[n];
} PXE_DB_RECEIVE_FILTERS;
```

## E.4.12 Station Address

This command is used to get current station and broadcast MAC addresses and, if supported, to change the current station MAC address.

### E.4.12.1 Issuing the Command

To issue a Station Address command, create a CDB and fill it in as shows in the table below:

| CDB Field | How to initialize the CDB structure for a Station Address command             |
|-----------|-------------------------------------------------------------------------------|
| OpCode    | PXE_OPCODE_STATION_ADDRESS                                                    |
| OpFlags   | Set as needed.                                                                |
| CPBsize   | sizeof(PXE_CPB_STATION_ADDRESS)                                               |
| DBsize    | sizeof(PXE_DB_STATION_ADDRESS)                                                |
| CPBaddr   | Address of PXE_CPB_STATION_ADDRESS structure.                                 |
| DBaddr    | Address of PXE_DB_STATION_ADDRESS structure.                                  |
| StatCode  | PXE_STATCODE_INITIALIZE                                                       |
| StatFlags | PXE_STATFLAGS_INITIALIZE                                                      |
| IFnum     | A valid interface number from zero to ( !PXE.IFcnt   (!PXE.IFcntExt << 8 ) ). |
| Control   | Set as needed.                                                                |



### E.4.12.2 OpFlags

To read current station and broadcast MAC addresses set the OpFlags field to:

- *PXE\_OPFLAGS\_STATION\_ADDRESS\_READ*

To change the current station to the address given in the CPB set the OpFlags field to:

- *PXE\_OPFLAGS\_STATION\_ADDRESS\_WRITE*

To reset the current station address back to the power on default, set the OpFlags field to:

- *PXE\_OPFLAGS\_STATION\_ADDRESS\_RESET*

### E.4.12.3 Preparing the CPB

To change the current station MAC address the *CDB.CPBsize* and *CDB.CPBaddr* fields must be set.

```
typedef struct s_pxe_cpb_station_address {
    // If supplied and supported, the current station MAC address
    // will be changed.

    PXE_MAC_ADDR StationAddr;
} PXE_CPB_STATION_ADDRESS;
```

### E.4.12.4 Waiting for the Command to Execute

Monitor the upper two bits (14 & 15) in the *CDB.StatFlags* field. Until these bits change to report *PXE\_STATFLAGS\_COMMAND\_COMPLETE* or *PXE\_STATFLAGS\_COMMAND\_FAILED*, the command has not been executed by the UNDI.

| StatFlags        | Reason                                              |
|------------------|-----------------------------------------------------|
| COMMAND_COMPLETE | Command completed successfully. DB is written.      |
| COMMAND_FAILED   | Command failed. StatCode field contains error code. |
| COMMAND_QUEUED   | Command has been queued.                            |
| INITIALIZE       | Command has been not executed or queued.            |

### E.4.12.5 Checking Command Execution Results

After command execution completes, either successfully or not, the *CDB.StatCode* field contains the result of the command execution.

| StatCode        | Reason                                                |
|-----------------|-------------------------------------------------------|
| SUCCESS         | Command completed successfully.                       |
| INVALID_CDB     | One of the CDB fields was not set correctly.          |
| INVALID_CPB     | One of the CPB fields was not set correctly.          |
| BUSY            | UNDI is already processing commands. Try again later. |
| QUEUE_FULL      | Command queue is full. Try again later.               |
| NOT_STARTED     | The UNDI is not started.                              |
| NOT_INITIALIZED | The UNDI is not initialized.                          |

continues on next page

Table E.31 – continued from previous page

|             |                                           |
|-------------|-------------------------------------------|
| UNSUPPORTED | The requested operation is not supported. |
|-------------|-------------------------------------------|

#### E.4.12.6 Before Using the DB

The DB is used to read the current station, broadcast and permanent station MAC addresses. The *CDB.DBsize* and *CDB.DBaddr* fields can be set to *PXE\_DBSIZE\_NOT\_USED* and *PXE\_DBADDR\_NOT\_USED* if these addresses do not need to be read.

```
typedef struct s_pxe_db_station_address {
    // Current station MAC address.
    PXE_MAC_ADDR StationAddr;

    // Station broadcast MAC address.
    PXE_MAC_ADDR BroadcastAddr;

    // Permanent station MAC address.
    PXE_MAC_ADDR PermanentAddr;
} PXE_DB_STATION_ADDRESS;
```

#### E.4.13 Statistics

This command is used to read and clear the NIC traffic statistics. Before using this command check to see if statistics is supported in the *!PXE.Implementation* flags.

##### E.4.13.1 Issuing the Command

To issue a Statistics command, create a CDB and fill it in as shown in the table below:

| CDB Field | How to initialize the CDB structure for a Statistics command                                |
|-----------|---------------------------------------------------------------------------------------------|
| OpCode    | PXE_OPCODE_STATISTICS                                                                       |
| OpFlags   | Set as needed.                                                                              |
| CPBsize   | PXE_CPBSIZE_NOT_USED                                                                        |
| DBsize    | sizeof(PXE_DB_STATISTICS)                                                                   |
| CPBaddr   | PXE_CPBADDR_NOT_USED                                                                        |
| DBaddr    | Address of PXE_DB_STATISTICS structure.                                                     |
| StatCode  | PXE_STATCODE_INITIALIZE                                                                     |
| StatFlags | PXE_STATFLAGS_INITIALIZE                                                                    |
| IFnum     | A valid interface number from zero to ( <i>!PXE.IFcnt</i>   ( <i>!PXE.IFcntExt</i> << 8) ). |
| Control   | Set as needed.                                                                              |

### E.4.13.2 OpFlags

To read the current statistics counters set the OpFlags field to:

```
PXE_OPFLAGS_STATISTICS_READ
```

To reset the current statistics counters set the OpFlags field to:

```
PXE_OPFLAGS_STATISTICS_RESET
```

### E.4.13.3 Waiting for the Command to Execute

Monitor the upper two bits (14 & 15) in the *CDB.StatFlags* field. Until these bits change to report *PXE\_STATFLAGS\_COMMAND\_COMPLETE* or *PXE\_STATFLAGS\_COMMAND\_FAILED*, the command has not been executed by the UNDI.

| StatFlags        | Reason                                              |
|------------------|-----------------------------------------------------|
| COMMAND_COMPLETE | Command completed successfully. DB is written.      |
| COMMAND_FAILED   | Command failed. StatCode field contains error code. |
| COMMAND_QUEUED   | Command has been queued.                            |
| INITIALIZE       | Command has been not executed or queued.            |

### E.4.13.4 Checking Command Execution Results

After command execution completes, either successfully or not, the *CDB.StatCode* field contains the result of the command execution.

| StatCode        | Reason                                                |
|-----------------|-------------------------------------------------------|
| SUCCESS         | Command completed successfully. DB is written.        |
| INVALID_CDB     | One of the CDB fields was not set correctly.          |
| BUSY            | UNDI is already processing commands. Try again later. |
| QUEUE_FULL      | Command queue is full. Try again later.               |
| NOT_STARTED     | The UNDI is not started.                              |
| NOT_INITIALIZED | The UNDI is not initialized.                          |
| UNSUPPORTED     | This command is not supported.                        |

### E.4.13.5 DB

Unsupported statistics counters will be zero filled by UNDI.

```
typedef struct s_pxe_db_statistics {
    // Bit field identifying what statistic data is collected by
    // the UNDI/NIC.
    // If bit 0x00 is set, Data[0x00] is collected.
    // If bit 0x01 is set, Data[0x01] is collected.
```

(continues on next page)

(continued from previous page)

```

// If bit 0x20 is set, Data[0x20] is collected.
// If bit 0x21 is set, Data[0x21] is collected.
// Etc.
PXE_UINT64 Supported;

// Statistic data.

PXE_UINT64 Data[64];
} PXE_DB_STATISTICS;

// Total number of frames received. Includes frames with errors
// and dropped frames.
#define PXE_STATISTICS_RX_TOTAL_FRAMES 0x00

// Number of valid frames received and copied into receive
// buffers.
#define PXE_STATISTICS_RX_GOOD_FRAMES 0x01

// Number of frames below the minimum length for the media.
// This would be <64 for ethernet.
#define PXE_STATISTICS_RX_UNDERSIZE_FRAMES 0x02

// Number of frames longer than the maximum length for the
// media. This would be >1500 for ethernet.
#define PXE_STATISTICS_RX_OVERSIZE_FRAMES 0x03

// Valid frames that were dropped because receive buffers
// were full.
#define PXE_STATISTICS_RX_DROPPED_FRAMES 0x04

// Number of valid unicast frames received and not dropped.
#define PXE_STATISTICS_RX_UNICAST_FRAMES 0x05

// Number of valid broadcast frames received and not dropped.
#define PXE_STATISTICS_RX_BROADCAST_FRAMES 0x06

// Number of valid multicast frames received and not dropped.
#define PXE_STATISTICS_RX_MULTICAST_FRAMES 0x07

// Number of frames w/ CRC or alignment errors.
#define PXE_STATISTICS_RX_CRC_ERROR_FRAMES 0x08

// Total number of bytes received. Includes frames with errors
// and dropped frames.
#define PXE_STATISTICS_RX_TOTAL_BYTES 0x09

// Transmit statistics.
#define PXE_STATISTICS_TX_TOTAL_FRAMES 0x0A
#define PXE_STATISTICS_TX_GOOD_FRAMES 0x0B
#define PXE_STATISTICS_TX_UNDERSIZE_FRAMES 0x0C
#define PXE_STATISTICS_TX_OVERSIZE_FRAMES 0x0D
#define PXE_STATISTICS_TX_DROPPED_FRAMES 0x0E

```

(continues on next page)

(continued from previous page)

```

#define PXE_STATISTICS_TX_UNICAST_FRAMES 0x0F
#define PXE_STATISTICS_TX_BROADCAST_FRAMES 0x10
#define PXE_STATISTICS_TX_MULTICAST_FRAMES 0x11
#define PXE_STATISTICS_TX_CRC_ERROR_FRAMES 0x12
#define PXE_STATISTICS_TX_TOTAL_BYTES 0x13

// Number of collisions detection on this subnet.
#define PXE_STATISTICS_COLLISIONS 0x14

// Number of frames destined for unsupported protocol.
#define PXE_STATISTICS_UNSUPPORTED_PROTOCOL 0x15

// Number of valid frames received that were duplicated.
#define PXE_STATISTICS_RX_DUPLICATED_FRAMES 0x16

// Number of encrypted frames received that failed to decrypt.
#define PXE_STATISTICS_RX_DECRYPT_ERROR_FRAMES 0x17

// Number of frames that failed to transmit after exceeding the
// retry limit.
#define PXE_STATISTICS_TX_ERROR_FRAMES 0x18

// Number of frames transmitted successfully after more than one
// attempt.
#define PXE_STATISTICS_TX_RETRY_FRAMES 0x19

```

## E.4.14 MCast IP To MAC

Translate a multicast IPv4 or IPv6 address to a multicast MAC address.

### E.4.14.1 Issuing the Command

To issue a MCast IP To MAC command, create a CDB and fill it in as shown in the table below:

| CDB Field | How to initialize the CDB structure for a MCast IP To MAC command             |
|-----------|-------------------------------------------------------------------------------|
| OpCode    | PXE_OPCODE_MCAST_IP_TO_MAC                                                    |
| OpFlags   | Set as needed.                                                                |
| CPBsize   | sizeof(PXE_CPB_MCAST_IP_TO_MAC)                                               |
| DBsize    | sizeof(PXE_DB_MCAST_IP_TO_MAC)                                                |
| CPBaddr   | Address of PXE_CPB_MCAST_IP_TO_MAC structure.                                 |
| Dbaddr    | Address of PXE_DB_MCAST_IP_TO_MAC structure.                                  |
| StatCode  | PXE_STATCODE_INITIALIZE                                                       |
| StatFlags | PXE_STATFLAGS_INITIALIZE                                                      |
| Ifnum     | A valid interface number from zero to ( !PXE.IFcnt   (!PXE.IFcntExt << 8 ) ). |
| Control   | Set as needed.                                                                |

#### E.4.14.2 OpFlags

To convert a multicast IP address to a multicast MAC address the UNDI needs to know the format of the IP address. Set one of these OpFlags to identify the format of the IP addresses in the CPB:

```
PXE_OPFLAGS_MCAST_IPV4_TO_MAC
PXE_OPFLAGS_MCAST_IPV6_TO_MAC
```

#### E.4.14.3 Preparing the CPB

Fill in an array of one or more multicast IP addresses. Be sure to set the *CDB.CPBsize* and *CDB.CPBaddr* fields accordingly.

```
typedef struct s_pxe_cpb_mcast_ip_to_mac {

    // Multicast IP address to be converted to multicast
    // MAC address.
    PXE_IP_ADDR IP[n];
}    PXE_CPB_MCAST_IP_TO_MAC;
```

#### E.4.14.4 Waiting for the Command to Execute

Monitor the upper two bits (14 & 15) in the *CDB.StatFlags* field. Until these bits change to report *PXE\_STATFLAGS\_COMMAND\_COMPLETE* or *PXE\_STATFLAGS\_COMMAND\_FAILED*, the command has not been executed by the UNDI.

| StatFlags        | Reason                                              |
|------------------|-----------------------------------------------------|
| COMMAND_COMPLETE | Command completed successfully. DB is written.      |
| COMMAND_FAILED   | Command failed. StatCode field contains error code. |
| COMMAND_QUEUED   | Command has been queued.                            |
| INITIALIZE       | Command has been not executed or queued.            |

#### E.4.14.5 Checking Command Execution Results

After command execution completes, either successfully or not, the *CDB.StatCode* field contains the result of the command execution.

| StatCode        | Reason                                                |
|-----------------|-------------------------------------------------------|
| SUCCESS         | Command completed successfully. DB is written.        |
| INVALID_CDB     | One of the CDB fields was not set correctly.          |
| INVALID_CPB     | One of the CPB fields was not set correctly.          |
| BUSY            | UNDI is already processing commands. Try again later. |
| QUEUE_FULL      | Command queue is full. Try again later.               |
| NOT_STARTED     | The UNDI is not started.                              |
| NOT_INITIALIZED | The UNDI is not initialized.                          |

#### E.4.14.6 Before Using the DB

The DB is where the multicast MAC addresses will be written.

```
typedef struct s_pxe_db_mcast_ip_to_mac {

    // Multicast MAC address.

    PXE_MAC_ADDR MAC[n];
}    PXE_DB_MCAST_IP_TO_MAC;
```

#### E.4.15 NvData

This command is used to read and write (if supported by NIC H/W) nonvolatile storage on the NIC. Nonvolatile storage could be EEPROM, FLASH or battery backed RAM.

##### E.4.15.1 Issuing the Command

To issue a NvData command, create a CDB and fill it in as shown in the table below:

| CDB Field | How to initialize the CDB structure for a NvData command                      |
|-----------|-------------------------------------------------------------------------------|
| OpCode    | PXE_OPCODE_NVDATA                                                             |
| OpFlags   | Set as needed.                                                                |
| CPBsize   | <i>sizeof(PXE_CPB_NVDATA)</i>                                                 |
| DBsize    | <i>sizeof(PXE_DB_NVDATA)</i>                                                  |
| CPBaddr   | Address of PXE_CPB_NVDATA structure.                                          |
| Dbaddr    | Address of PXE_DB_NVDATA structure.                                           |
| StatCode  | PXE_STATCODE_INITIALIZE                                                       |
| StatFlags | PXE_STATFLAGS_INITIALIZE                                                      |
| Ifnum     | A valid interface number from zero to ( !PXE.IFcnt   (!PXE.IFcntExt << 8 ) ). |
| Control   | Set as needed.                                                                |

##### E.4.15.2 Preparing the CPB

There are two types of nonvolatile data CPBs, one for sparse updates and one for bulk updates. Sparse updates allow updating of single nonvolatile storage items. Bulk updates always update all nonvolatile storage items. Check the *!PXE.Implementation* flags to see which type of nonvolatile update is supported by this UNDI and network device.

If you do not need to update the nonvolatile storage set the *CDB.CPBsize* and *CDB.CPBaddr* fields to *PXE\_CPBSIZE\_NOT\_USED* and *PXE\_CPBADDR\_NOT\_USED*.

#### E.4.15.2.1 Sparse NvData CPB

```
typedef struct s_pxe_cpb_nvdata_sparse {
    // NvData item list. Only items in this list will be updated.

    struct {

        // Nonvolatile storage address to be changed.
        PXE_UINT32 Addr;

        // Data item to write into above storage address.
        union {
            PXE_UINT8 Byte;
            PXE_UINT16 Word;
            PXE_UINT32 Dword;
        } Data;
    } Item[n];
} PXE_CPB_NVDATA_SPARSE;
```

#### E.4.15.2.2 Bulk NvData CPB

```
// When using bulk update, the size of the CPB structure must be
// the same size as the nonvolatile NIC storage.

typedef union u_pxe_cpb_nvdata_bulk {

    // Array of byte-wide data items.
    PXE_UINT8 Byte[n];

    // Array of word-wide data items.
    PXE_UINT16 Word[n];

    // Array of dword-wide data items.
    PXE_UINT32 Dword[n];
} PXE_CPB_NVDATA_BULK;
```

#### E.4.15.3 Waiting for the Command to Execute

Monitor the upper two bits (14 & 15) in the *CDB.StatFlags* field. Until these bits change to report *PXE\_STATFLAGS\_COMMAND\_COMPLETE* or *PXE\_STATFLAGS\_COMMAND\_FAILED*, the command has not been executed by the UEFI.

| StatFlags        | Reason                                                                                     |
|------------------|--------------------------------------------------------------------------------------------|
| COMMAND_COMPLETE | Command completed successfully. Nonvolatile data is updated from CPB and/or written to DB. |
| COMMAND_FAILED   | Command failed. StatCode field contains error code.                                        |
| COMMAND_QUEUED   | Command has been queued.                                                                   |
| INITIALIZE       | Command has been not executed or queued.                                                   |



#### E.4.15.4 Checking Command Execution Results

After command execution completes, either successfully or not, the *CDB.StatCode* field contains the result of the command execution.

| StatCode        | Reason                                                                                     |
|-----------------|--------------------------------------------------------------------------------------------|
| SUCCESS         | Command completed successfully. Nonvolatile data is updated from CPB and/or written to DB. |
| INVALID_CDB     | One of the CDB fields was not set correctly.                                               |
| INVALID_CPB     | One of the CPB fields was not set correctly.                                               |
| BUSY            | UNDI is already processing commands. Try again later.                                      |
| QUEUE_FULL      | Command queue is full. Try again later.                                                    |
| NOT_STARTED     | The UNDI is not started.                                                                   |
| NOT_INITIALIZED | The UNDI is not initialized.                                                               |
| UNSUPPORTED     | Requested operation is unsupported.                                                        |

##### E.4.15.4.1 DB

Check the width and number of nonvolatile storage items. This information is returned by the Get Init Info command.

```
typedef struct s_pxe_db_nvdata {
    // Arrays of data items from nonvolatile storage.
    union {
        // Array of byte-wide data items.
        PXE_UINT8 Byte[n];

        // Array of word-wide data items.
        PXE_UINT16 Word[n];

        // Array of dword-wide data items.
        PXE_UINT32 Dword[n];
    } Data;
} PXE_DB_NVDATA;
```

#### E.4.16 Get Status

This command returns the current interrupt status and/or the transmitted buffer addresses and the current media status. If the current interrupt status is returned, pending interrupts will be acknowledged by this command. Transmitted buffer addresses that are written to the DB are removed from the transmitted buffer queue.

This command may be used in a polled fashion with external interrupts disabled.

### E.4.16.1 Issuing the Command

To issue a Get Status command, create a CDB and fill it in as shown in the table below:

| CDB Field | How to initialize the CDB structure for a Get Status command                      |
|-----------|-----------------------------------------------------------------------------------|
| OpCode    | PXE_OPCODE_GET_STATUS                                                             |
| OpFlags   | Set as needed.                                                                    |
| CPBsize   | PXE_CPBSIZE_NOT_USED                                                              |
| DBsize    | Sizeof(PXE_DB_GET_STATUS)                                                         |
| CPBaddr   | PXE_CPBADDR_NOT_USED                                                              |
| DBaddr    | Address of PXE_DB_GET_STATUS structure.                                           |
| StatCode  | PXE_STATCODE_INITIALIZE                                                           |
| StatFlags | PXE_STATFLAGS_INITIALIZE                                                          |
| IFnum     | A valid interface number from zero to ( $!PXE.IFcnt \mid (!PXE.IFcntExt << 8)$ ). |
| Control   | Set as needed.                                                                    |

#### E.4.16.1.1 Setting OpFlags

Set one or a combination of the OpFlags below to return the interrupt status and/or the transmitted buffer addresses and/or the media status.

```
PXE_OPFLAGS_GET_INTERRUPT_STATUS
PXE_OPFLAGS_GET_TRANSMITTED_BUFFERS
PXE_OPFLAGS_GET_MEDIA_STATUS
```

### E.4.16.2 Waiting for the Command to Execute

Monitor the upper two bits (14 & 15) in the *CDB.StatFlags* field. Until these bits change to report *PXE\_STATFLAGS\_COMMAND\_COMPLETE* or *PXE\_STATFLAGS\_COMMAND\_FAILED*, the command has not been executed by the UNDI.

| StatFlags        | Reason                                                           |
|------------------|------------------------------------------------------------------|
| COMMAND_COMPLETE | Command completed successfully. StatFlags and/or DB are updated. |
| COMMAND_FAILED   | Command failed. StatCode field contains error code.              |
| COMMAND_QUEUED   | Command has been queued.                                         |
| INITIALIZE       | Command has been not executed or queued.                         |

### E.4.16.3 Checking Command Execution Results

After command execution completes, either successfully or not, the *CDB.StatCode* field contains the result of the command execution.

| StatCode    | Reason                                                           |
|-------------|------------------------------------------------------------------|
| SUCCESS     | Command completed successfully. StatFlags and/or DB are updated. |
| INVALID_CDB | One of the CDB fields was not set correctly.                     |
| BUSY        | UNDI is already processing commands. Try again later.            |
| QUEUE_FULL  | Command queue is full. Try again later.                          |

continues on next page

Table E.43 – continued from previous page

|                 |                              |
|-----------------|------------------------------|
| NOT_STARTED     | The UNDI is not started.     |
| NOT_INITIALIZED | The UNDI is not initialized. |

#### E.4.16.4 StatFlags

If the command completes successfully and the *PXE\_OPFLAGS\_GET\_INTERRUPT\_STATUS* OpFlag was set in the CDB, the current interrupt status is returned in the *CDB.StatFlags* field and any pending interrupts will have been cleared.

```
PXE_STATFLAGS_GET_STATUS_RECEIVE
PXE_STATFLAGS_GET_STATUS_TRANSMIT
PXE_STATFLAGS_GET_STATUS_COMMAND
PXE_STATFLAGS_GET_STATUS_SOFTWARE
```

The StatFlags above may not map directly to external interrupt signals. For example: Some NICs may combine both the receive and transmit interrupts to one external interrupt line. When a receive and/or transmit interrupt occurs, use the Get Status to determine which type(s) of interrupt(s) occurred.

This flag is set if the transmitted buffer queue is empty. This flag will be set if all transmitted buffer addresses get written into the DB.

```
PXE_STATFLAGS_GET_STATUS_TXBUF_QUEUE_EMPTY
```

This flag is set if no transmitted buffer addresses were written into the DB.

```
PXE_STATFLAGS_GET_STATUS_NO_TXBUFS_WRITTEN
```

This flag is set if there is no media present.

```
PXE_STATFLAGS_GET_STATUS_NO_MEDIA
```

#### E.4.16.5 Using the DB

When reading the transmitted buffer addresses there should be room for at least one 64-bit address in the DB. Once a complete transmitted buffer address is written into the DB, the address is removed from the transmitted buffer queue. If the transmitted buffer queue is full, attempts to use the Transmit command will fail.

```
#pragma pack(1)
typedef struct s_pxe_db_get_status {

    // Length of next receive frame (header + data). If this is
    // zero, there is no next receive frame available.
    PXE_UINT32 RxFrameLen;

    // Reserved, set to zero.

    PXE_UINT32 reserved;

    // Addresses of transmitted buffers that need to be recycled.

    PXE_UINT64 xBuffer[n];
```

(continues on next page)

(continued from previous page)

```

}    PXE_DB_GET_STATUS;
#pragma pack()

```

## E.4.17 Fill Header

This command is used to fill the media header(s) in transmit packet(s).

### E.4.17.1 Issuing the Command

To issue a Fill Header command, create a CDB and fill it in as shown in the table below:

| CDB Field | How to initialize the CDB structure for a Fill Header command                               |
|-----------|---------------------------------------------------------------------------------------------|
| OpCode    | PXE_OPCODE_FILL_HEADER                                                                      |
| OpFlags   | Set as needed.                                                                              |
| CPBsize   | PXE_CPB_FILL_HEADER                                                                         |
| DBsize    | PXE_DBSIZE_NOT_USED                                                                         |
| CPBaddr   | Address of a PXE_CPB_FILL_HEADER structure.                                                 |
| DBaddr    | PXE_DBADDR_NOT_USED                                                                         |
| StatCode  | PXE_STATCODE_INITIALIZE                                                                     |
| StatFlags | PXE_STATFLAGS_INITIALIZE                                                                    |
| IFnum     | A valid interface number from zero to ( <i>!PXE.IFcnt</i>   ( <i>!PXE.IFcntExt</i> << 8) ). |
| Control   | Set as needed.                                                                              |

### E.4.17.2 OpFlags

Select one of the OpFlags below so the UNDI knows what type of CPB is being used.

```

PXE_OPFLAGS_FILL_HEADER_WHOLE
PXE_OPFLAGS_FILL_HEADER_FRAGMENTED

```

### E.4.17.3 Preparing the CPB

If multiple frames per command are supported (see *!PXE.Implementation* flags), multiple CPBs can be packed together. The *CDB.CPBsize* field lets the UNDI know how many CPBs are packed together.

### E.4.17.4 Nonfragmented Frame

```

#pragma pack(1)
typedef struct s_pxe_cpb_fill_header {

    // Source and destination MAC addresses. These will be copied
    // into the media header without doing byte swapping.
    PXE_MAC_ADDR SrcAddr;
    PXE_MAC_ADDR DestAddr;

    // Address of first byte of media header. The first byte of

```

(continues on next page)

(continued from previous page)

```

// packet data follows the last byte of the media header.
PXE_UINT64 MediaHeader;

// Length of packet data in bytes (not including the media
// header).
PXE_UINT32 PacketLen;

// Protocol type. This will be copied into the media header
// without doing byte swapping. Protocol type numbers can be
// obtained from the Assigned Numbers RFC 3232.
PXE_UINT16 Protocol;

// Length of the media header in bytes.
PXE_UINT16 MediaHeaderLen;
} PXE_CPB_FILL_HEADER;
#pragma pack()

#define PXE_PROTOCOL_ETHERNET_IP 0x0800
#define PXE_PROTOCOL_ETHERNET_ARP 0x0806

```

#### E.4.17.5 Fragmented Frame

```

#pragma pack(1)
typedef struct s_pxe_cpb_fill_header_fragmented {

// Source and destination MAC addresses. These will be copied
// into the media header without doing byte swapping.
PXE_MAC_ADDR SrcAddr;
PXE_MAC_ADDR DestAddr;

// Length of packet data in bytes (not including the media
// header).

PXE_UINT32 PacketLen;
// Protocol type. This will be copied into the media header
// without doing byte swapping. Protocol type numbers can be
// obtained from the Assigned Numbers RFC 3232.
PXE_MEDIA_PROTOCOL Protocol;

// Length of the media header in bytes.
PXE_UINT16 MediaHeaderLen;

// Number of packet fragment descriptors.
PXE_UINT16 FragCnt;

// Reserved, must be set to zero.
PXE_UINT16 reserved;

// Array of packet fragment descriptors. The first byte of the
// media header is the first byte of the first fragment.

```

(continues on next page)

(continued from previous page)

```

struct {
    // Address of this packet fragment.
    PXE_UINT64 FragAddr;
    // Length of this packet fragment.
    PXE_UINT32 FragLen;

    // Reserved, must be set to zero.
    PXE_UINT32 reserved;
} FragDesc[n];
} PXE_CPB_FILL_HEADER_FRAGMENTED;
#pragma pack()

```

#### E.4.17.6 Waiting for the Command to Execute

Monitor the upper two bits (14 & 15) in the *CDB.StatFlags* field. Until these bits change to report *PXE\_STATFLAGS\_COMMAND\_COMPLETE* or *PXE\_STATFLAGS\_COMMAND\_FAILED*, the command has not been executed by the UNDI.

| StatFlags        | Reason                                                      |
|------------------|-------------------------------------------------------------|
| COMMAND_COMPLETE | Command completed successfully. Frame is ready to transmit. |
| COMMAND_FAILED   | Command failed. StatCode field contains error code.         |
| COMMAND_QUEUED   | Command has been queued.                                    |
| INITIALIZE       | Command has been not executed or queued.                    |

#### E.4.17.7 Checking Command Execution Results

After command execution completes, either successfully or not, the *CDB.StatCode* field contains the result of the command execution.

| StatCode        | Reason                                                      |
|-----------------|-------------------------------------------------------------|
| SUCCESS         | Command completed successfully. Frame is ready to transmit. |
| INVALID_CDB     | One of the CDB fields was not set correctly.                |
| INVALID_CPB     | One of the CPB fields was not set correctly.                |
| BUSY            | UNDI is already processing commands. Try again later.       |
| QUEUE_FULL      | Command queue is full. Try again later.                     |
| NOT_STARTED     | The UNDI is not started.                                    |
| NOT_INITIALIZED | The UNDI is not initialized.                                |

## E.4.18 Transmit

The Transmit command is used to place a packet into the transmit queue. The data buffers given to this command are to be considered locked and the application or universal network driver loses the ownership of those buffers and must not free or relocate them until the ownership returns.

When the packets are transmitted, a transmit complete interrupt is generated (if interrupts are disabled, the transmit interrupt status is still set and can be checked using the Get Status command).

Some UNDI implementations and network adapters support transmitting multiple packets with one transmit command. If this feature is supported, multiple transmit CPBs can be linked in one transmit command.

Though all UNDI implementations support fragmented frames, the same cannot be said for all network devices or protocols. If a fragmented frame CPB is given to UNDI and the network device does not support fragmented frames (see *!PXE.Implementation* flags), the UNDI will have to copy the fragments into a local buffer before transmitting.

### E.4.18.1 Issuing the Command

To issue a Transmit command, create a CDB and fill it in as shown in the table below:

| CDB Field | How to initialize the CDB structure for a Transmit command                                  |
|-----------|---------------------------------------------------------------------------------------------|
| OpCode    | PXE_OPCODE_TRANSMIT                                                                         |
| OpFlags   | Set as needed.                                                                              |
| CPBsize   | <i>sizeof(PXE_CPB_TRANSMIT)</i>                                                             |
| DBsize    | PXE_DBSIZE_NOT_USED                                                                         |
| CPBaddr   | Address of a PXE_CPB_TRANSMIT structure.                                                    |
| DBaddr    | PXE_DBADDR_NOT_USED                                                                         |
| StatCode  | PXE_STATCODE_INITIALIZE                                                                     |
| StatFlags | PXE_STATFLAGS_INITIALIZE                                                                    |
| IFnum     | A valid interface number from zero to ( <i>!PXE.IFcnt</i>   ( <i>!PXE.IFcntExt</i> << 8) ). |
| Control   | Set as needed.                                                                              |

### E.4.18.2 OpFlags

Check the *!PXE.Implementation* flags to see if the network device supports fragmented packets. Select one of the OpFlags below so the UNDI knows what type of CPB is being used.

```
PXE_OPFLAGS_TRANSMIT_WHOLE
PXE_OPFLAGS_TRANSMIT_FRAGMENTED
```

In addition to selecting whether or not fragmented packets are being given, S/W UNDI needs to know if it should block until the packets are transmitted. H/W UNDI cannot block, these two OpFlag settings have no effect when used with H/W UNDI.

```
PXE_OPFLAGS_TRANSMIT_BLOCK
PXE_OPFLAGS_TRANSMIT_DONT_BLOCK
```

### E.4.18.3 Preparing the CPB

If multiple frames per command are supported (see *!PXE.Implementation* flags), multiple CPBs can be packed together. The *CDB.CPBsize* field lets the UNDI know how many frames are to be transmitted.

### E.4.18.4 Nonfragmented Frame

```
#pragma pack(1)
typedef struct s_pxe_cpb_transmit {

    // Address of first byte of frame buffer. This is also the
    // first byte of the media header. This address must be a
    // processor-based address for S/W UNDI and a device-based
    // address for H/W UNDI.
    PXE_UINT64 FrameAddr;

    // Length of the data portion of the frame buffer in bytes. Do
    // not include the length of the media header.
    PXE_UINT32 DataLen;

    // Length of the media header in bytes.
    PXE_UINT16 MediaheaderLen;

    // Reserved, must be zero.
    PXE_UINT16 reserved;
}    PXE_CPB_TRANSMIT;
#pragma pack()
```

### E.4.18.5 Fragmented Frame

```
#pragma pack(1)
typedef struct s_pxe_cpb_transmit_fragments {

    // Length of packet data in bytes (not including the media
    // header).
    PXE_UINT32 FrameLen;

    // Length of the media header in bytes.
    PXE_UINT16 MediaheaderLen;

    // Number of packet fragment descriptors.
    PXE_UINT16 FragCnt;

    // Array of frame fragment descriptors. The first byte of the
    // first fragment is also the first byte of the media header.
    struct {
        // Address of this frame fragment. This address must be a
        // processor-based address for S/W UNDI and a device-based
        // address for H/W UNDI.
        PXE_UINT64 FragAddr;
```

(continues on next page)



(continued from previous page)

```

// Length of this frame fragment.
PXE_UINT32 FragLen;

// Reserved, must be set to zero.
PXE_UINT32 reserved;
} FragDesc[n];
} PXE_CPB_TRANSMIT_FRAGMENTS;
#pragma pack()

```

#### E.4.18.6 Waiting for the Command to Execute

Monitor the upper two bits (14 & 15) in the *CDB.StatFlags* field. Until these bits change to report *PXE\_STATFLAGS\_COMMAND\_COMPLETE* or *PXE\_STATFLAGS\_COMMAND\_FAILED*, the command has not been executed by the UNDI.

| StatFlags        | Reason                                                                                              |
|------------------|-----------------------------------------------------------------------------------------------------|
| COMMAND_COMPLETE | Command completed successfully. Use the Get Status command to see when frame buffers can be reused. |
| COMMAND_FAILED   | Command failed. StatCode field contains error code.                                                 |
| COMMAND_QUEUED   | Command has been queued.                                                                            |
| INITIALIZE       | Command has been not executed or queued.                                                            |

#### E.4.18.7 Checking Command Execution Results

After command execution completes, either successfully or not, the *CDB.StatCode* field contains the result of the command execution.

| StatCode        | Reason                                                                                              |
|-----------------|-----------------------------------------------------------------------------------------------------|
| SUCCESS         | Command completed successfully. Use the Get Status command to see when frame buffers can be reused. |
| INVALID_CDB     | One of the CDB fields was not set correctly.                                                        |
| INVALID_CPB     | One of the CPB fields was not set correctly.                                                        |
| BUSY            | UNDI is already processing commands. Try again later.                                               |
| QUEUE_FULL      | Command queue is full. Wait for queued commands to complete. Try again later.                       |
| BUFFER_FULL     | Transmit buffer is full. Call Get Status command to empty buffer.                                   |
| NOT_STARTED     | The UNDI is not started.                                                                            |
| NOT_INITIALIZED | The UNDI is not initialized.                                                                        |

#### E.4.19 Receive

When the network adapter has received a frame, this command is used to copy the frame into driver/application storage. Once a frame has been copied, it is removed from the receive queue.

### E.4.19.1 Issuing the Command

To issue a Receive command, create a CDB and fill it in as shown in the table below:

| CDB Field | How to initialize the CDB structure for a Receive command                         |
|-----------|-----------------------------------------------------------------------------------|
| OpCode    | PXE_OPCODE_RECEIVE                                                                |
| OpFlags   | Set as needed.                                                                    |
| CPBsize   | sizeof(PXE_CPB_RECEIVE)                                                           |
| DBsize    | sizeof(PXE_DB_RECEIVE)                                                            |
| CPBaddr   | Address of a PXE_CPB_RECEIVE structure.                                           |
| DBaddr    | Address of a PXE_DB_RECEIVE structure.                                            |
| StatCode  | PXE_STATCODE_INITIALIZE                                                           |
| StatFlags | PXE_STATFLAGS_INITIALIZE                                                          |
| IFnum     | A valid interface number from zero to ( $!PXE.IFcnt \mid (!PXE.IFcntExt << 8)$ ). |
| Control   | Set as needed.                                                                    |

### E.4.19.2 Preparing the CPB

If multiple frames per command are supported (see *!PXE.Implementation* flags), multiple CPBs can be packed together. For each complete received frame, a receive buffer large enough to contain the entire unfragmented frame needs to be described in the CPB. Note that if a smaller than required buffer is provided, only a portion of the packet is received into the buffer, and the remainder of the packet is lost. Subsequent attempts to receive the same packet with a corrected (larger) buffer will be unsuccessful, because the packet will have been flushed from the queue.

```
#pragma pack(1)
typedef struct s_pxe_cpb_receive {

    // Address of first byte of receive buffer. This is also the
    // first byte of the frame header. This address must be a
    // processor-based address for S/W UNDI and a device-based
    // address for H/W UNDI.

    PXE_UINT64 BufferAddr;

    // Length of receive buffer. This must be large enough to hold
    // the received frame (media header + data). If the length of
    // smaller than the received frame, data will be lost.
    PXE_UINT32 BufferLen;

    // Reserved, must be set to zero.
    PXE_UINT32 reserved;
} PXE_CPB_RECEIVE;
#pragma pack()
```

### E.4.19.3 Waiting for the Command to Execute

Monitor the upper two bits (14 & 15) in the *CDB.StatFlags* field. Until these bits change to report *PXE\_STATFLAGS\_COMMAND\_COMPLETE* or *PXE\_STATFLAGS\_COMMAND\_FAILED*, the command has not been executed by the UNDI.

| StatFlags        | Reason                                                             |
|------------------|--------------------------------------------------------------------|
| COMMAND_COMPLETE | Command completed successfully. Frames received and DB is written. |
| COMMAND_FAILED   | Command failed. StatCode field contains error code.                |
| COMMAND_QUEUED   | Command has been queued.                                           |
| INITIALIZE       | Command has been not executed or queued.                           |

### E.4.19.4 Checking Command Execution Results

After command execution completes, either successfully or not, the *CDB.StatCode* field contains the result of the command execution.

| StatCode        | Reason                                                                        |
|-----------------|-------------------------------------------------------------------------------|
| SUCCESS         | Command completed successfully. Frames received and DB is written.            |
| INVALID_CDB     | One of the CDB fields was not set correctly.                                  |
| INVALID_CPB     | One of the CPB fields was not set correctly.                                  |
| BUSY            | UNDI is already processing commands. Try again later.                         |
| QUEUE_FULL      | Command queue is full. Wait for queued commands to complete. Try again later. |
| NO_DATA         | Receive buffers are empty.                                                    |
| NOT_STARTED     | The UNDI is not started.                                                      |
| NOT_INITIALIZED | The UNDI is not initialized.                                                  |

### E.4.19.5 Using the DB

If multiple frames per command are supported (see *!PXE.Implementation* flags), multiple DBs can be packed together.

```
#pragma pack(1)
typedef struct s_pxe_db_receive {

    // Source and destination MAC addresses from media header.
    PXE_MAC_ADDR SrcAddr;
    PXE_MAC_ADDR DestAddr;

    // Length of received frame. May be larger than receive buffer
    // size. The receive buffer will not be overwritten. This is
    // how to tell if data was lost because the receive buffer was
    // too small.
    PXE_UINT32 FrameLen;

    // Protocol type from media header.
    PXE_PROTOCOL Protocol;

    // Length of media header in received frame.
    PXE_UINT16 MediaHeaderLen;
}
```

(continues on next page)

(continued from previous page)

```
// Type of receive frame.
PXE_FRAME_TYPE Type;

// Reserved, must be zero.
PXE_UINT8 reserved[7];
} PXE_DB_RECEIVE;
#pragma pack()
```

## E.4.20 PXE 2.1 specification wire protocol clarifications

The Preboot Execution Environment (PXE) Version 2.1 specification was published in September 1999. Since then, this specification has not been maintained or updated for a new version. For adapting the IPv6 stack, the definition of an IPv6-based PXE process has been described in UEFI specification since version 2.2. Other clarifications for the IPv4-based PXE process defined in the PXE 2.1 specification are provided in this section.

### E.4.20.1 Issue #1-time-outs

Where the PXE 2.1 specification reads:

DHCP Discover will be retried four times. The four timeouts are 4, 8, 16 and 32 seconds respectively.

If a DHCPOFFER is received without an Option #60 tag “PXEClient”, DHCP Discover will be retried on the 4-and 8-second timeouts in an attempt to receive a PXE response.

Because of spanning tree algorithms in routers, the behavior should be as follows:

DHCP Discover will be retried four times. The four timeouts are 4, 8, 16 and 32 seconds respectively.

This process could be iterated three times.

If a DHCPOFFER is received without an Option #60 tag “PXEClient”, DHCP Discover will be retried on the 4-and 8-second timeouts in an attempt to receive a PXE response.

### E.4.20.2 Issue #2 - siaddr/option 54 precedence

Where the PXE 2.1 specification reads:

Boot server IP address (Read from the DHCP option 54 (server identifier), if not found, use the siaddr field.)

The behavior should be reversed, namely:

Ascertain the Boot server IP address from siaddr field. If not found, use the value in the DHCP option 54 (server identifier).

### E.4.20.3 Issue #3 - PXE Vendor Options Existence

The PXE 2.1 specification is ambiguous about whether the following PXE Vendor Options need to be provided in DHCP messages. These options are marked as “Required” in Table 2-1 “PXE DHCP Options (Full List)”, but other parts of the specification state that these options may not be supplied in certain condition.

This section clarifies the existence of these PXE Vendor Options:

#### 1. PXE\_DISCOVERY\_CONTROL (Tag 6)

Where the PXE 2.1 specification reads:

- Required, Note #3
- If this tag is not supplied all bits assumed to be 0.

The behavior should be clarified as:

- This tag is not mandatory required. If not supplied, all bits are assumed to be 0.

## 2. **PXE\_BOOT\_SERVERS (Tag 8)**

Where the PXE 2.1 specification reads:

- Required for PXE client. Note #3
- PXE\_DISCOVERY\_CONTROL (Tag 6), bit 2 = If set, only use/accept servers in PXE\_BOOT\_SERVERS.

The behavior should be clarified as:

- This tag is required only if bit 2 of PXE\_DISCOVERY\_CONTROL (Tag 6) is set.

## 3. **PXE\_BOOT\_MENU (Tag 9)**

Where the PXE 2.1 specification reads:

- Required, Note #4
- Note #4: These options define the information, if any, displayed by the client during a network boot.

The behavior should be clarified as:

- This tag is required only if the PXE client wants to display boot menu information during a network boot.

## 4. **PXE\_CREDENTIAL\_TYPES (Tag 12)**

Where the PXE 2.1 specification reads:

- Required for security. Note #5
- This option is required for security requests and acknowledges between the client and the server.

The behavior should be clarified as:

- This tag is not required if PXE client does not apply security requests.

## 5. **PXE\_BOOT\_ITEM (Tag 71)**

Where the PXE 2.1 specification reads:

- Required. Note #6
- If this tag is missing, type 0 and layer 0 is assumed.\*\*

The behavior should be clarified as:

- This tag is not mandatory required. If not supplied, type 0 and layer 0 is assumed.

## 6. **Vendor Options (Tag 43)**

The PXE 2.1 specification is not clear whether this option is required.  
The behavior should be clarified as:

- Vendor Options (Tag 43) is required only if encapsulated PXE options need be supplied.